

Basic Lisp Techniques

David J. Cooper, Jr.

February 14, 2011

© Copyright 2011, Franz Inc. and David J. Cooper, Jr.

Foreword¹

Computers, and the software applications that power them, permeate every facet of our daily lives. From groceries to airline reservations to dental appointments, our reliance on technology is all-encompassing. And, it's not enough. Every day, our expectations of technology and software increase:

- smart appliances that can be controlled via the internet
- better search engines that generate information we actually want
- voice-activated laptops
- cars that know exactly where to go

The list is endless. Unfortunately, there is not an endless supply of programmers and developers to satisfy our insatiable appetites for new features and gadgets. Every day, hundreds of magazine and on-line articles focus on the time and people resources needed to support future technological expectations. Further, the days of unlimited funding are over. Investors want to see results, fast.

Common Lisp (CL) is one of the few languages and development options that can meet these challenges. Powerful, flexible, changeable on the fly — increasingly, CL is playing a leading role in areas with complex problem-solving demands. Engineers in the fields of bioinformatics, scheduling, data mining, document management, B2B, and E-commerce have all turned to CL to complete their applications on time and within budget. CL, however, no longer just appropriate for the most complex problems. Applications of modest complexity, but with demanding needs for fast development cycles and customization, are also ideal candidates for CL.

Other languages have tried to mimic CL, with limited success. Perl, Python, Java, C++, C# — they all incorporate some of the features that give Lisp its power, but their implementations tend to be brittle.

¹this Foreword was authored by Franz Inc.

The purpose of this book is to showcase the features that make CL so much better than these imitators, and to give you a “quick-start” guide for using Common Lisp as a development environment. If you are an experienced programmer in languages other than Lisp, this guide gives you all the tools you need to begin writing Lisp applications. If you’ve had some exposure to Lisp in the past, this guide will help refresh those memories and shed some new light on CL for you.

But be careful, Lisp can be addicting! This is why many Fortune 500 companies will use nothing else on their 24/7, cutting-edge, mission-critical applications. After reading this book, trying our software, and experiencing a 3 to 10 times increase in productivity, we believe you will feel the same way.

Contents

Foreword	iii
1 Introduction	1
1.1 The Past, Present, and Future of Common Lisp	1
1.1.1 Lisp Yesterday	1
1.1.2 Lisp Today	2
1.1.3 Lisp Tomorrow	2
1.2 Convergence of Hardware and Software	3
1.3 The CL Model of Computation	4
2 Operating a CL Development Environment	5
2.1 Installing a CL Environment	6
2.2 Running CL in a Shell Window	6
2.2.1 Starting CL from a Terminal Window	6
2.2.2 Stopping CL from a Terminal Window	7
2.3 Running CL inside a Text Editor	8
2.3.1 A Note on Emacs and Text Editors	9
2.3.2 Emacs Terminology	9
2.3.3 Starting, Stopping, and Working With CL inside an Emacs Shell	10
2.4 Running CL as a subprocess of Emacs	11
2.4.1 Starting the CL subprocess within Emacs	11
2.4.2 Working with CL as an Emacs subprocess	12
2.4.3 Compiling and Loading a File from an Emacs buffer .	13
2.5 Integrated Development Environment	14
2.6 The User Init File	15
2.7 Using CL as a scripting language	16
2.8 Debugging in CL	16
2.8.1 Common debugger commands	17

2.8.2	Interpreted vs Compiled Code	18
2.8.3	Use of (break) and C-c to interrupt	18
2.8.4	Profiling	18
2.9	Developing Programs and Applications in CL	19
2.9.1	A Layered Approach	19
2.9.2	Compiling and Loading your Project	19
2.9.3	Creating an Application “Fasl” File	20
2.9.4	Creating an Image File	21
2.9.5	Building Runtime Images	21
2.9.6	Using an Application Init File	22
3	The CL Language	23
3.1	Overview of CL and its Syntax	23
3.1.1	Evaluation of Arguments to a Function	25
3.1.2	Lisp Syntax Simplicity	25
3.1.3	Turning Off Evaluation	26
3.1.4	Fundamental CL Data Types	27
3.1.5	Functions	30
3.1.6	Global and Local Variables	31
3.2	The List as a Data Structure	34
3.2.1	Accessing the Elements of a List	34
3.2.2	The “Rest” of the Story	35
3.2.3	The Empty List	35
3.2.4	Are You a List?	36
3.2.5	The conditional If	36
3.2.6	Length of a List	37
3.2.7	Member of a List	37
3.2.8	Getting Part of a List	38
3.2.9	Appending Lists	39
3.2.10	Adding Elements to a List	39
3.2.11	Removing Elements from a List	40
3.2.12	Sorting Lists	40
3.2.13	Treating a List as a Set	41
3.2.14	Mapping a Function to a List	41
3.2.15	Property Lists	42
3.3	Control of Execution	42
3.3.1	If	42
3.3.2	When	43
3.3.3	Logical Operators	43
3.3.4	Cond	44

3.3.5	Case	45
3.3.6	Iteration	46
3.4	Functions as Objects	46
3.4.1	Named Functions	46
3.4.2	Functional Arguments	47
3.4.3	Anonymous Functions	48
3.4.4	Optional Arguments	48
3.4.5	Keyword Arguments	49
3.5	Input, Output, Streams, and Strings	49
3.5.1	Read	50
3.5.2	Print and Prin1	50
3.5.3	Princ	51
3.5.4	Format	51
3.5.5	Pathnames	52
3.5.6	File Input and Output	53
3.6	Hash Tables, Arrays, Structures, and Classes	53
3.6.1	Hash Tables	54
3.6.2	Arrays	55
3.6.3	Structures	55
3.6.4	Classes and Methods	55
3.7	Packages	57
3.7.1	Importing and Exporting Symbols	57
3.7.2	The Keyword Package	58
3.8	Common Stumbling Blocks	58
3.8.1	Quotes	58
3.8.2	Function Argument Lists	59
3.8.3	Symbols vs. Strings	59
3.8.4	Equality	61
3.8.5	Distinguishing Macros from Functions	62
3.8.6	Operations that cons	63
4	Interfaces	65
4.1	The Operating System	65
4.1.1	System Calls	65
4.1.2	Environment Variables	66
4.2	Foreign Function Interface	66
4.3	CORBA	67
4.4	TCP/IP Sockets	67
4.5	Databases	69
4.6	Web-serving	70

4.6.1	Publishing Static Pages	71
4.6.2	Publishing Dynamic Pages	72
4.6.3	Query Parameters	72
4.6.4	Form Parameters	73
4.6.5	Multipart Form Data and File Uploads	74
4.6.6	Development and Production Modes	74
4.6.7	Other Features	74
4.7	Embedded Languages	75
4.8	Conclusion	75
5	The Squeakymail Example	77
5.1	Requirements and Design Overview	77
5.2	Main Application File	78
5.3	Working With Global State	79
5.4	Background Polling	79
5.5	Serving the Web Pages	80
5.6	Composing and Sending Mail	83
5.7	Deleting Messages	84
5.8	Starting the Webserver	84
5.9	Message Data Structure	85
5.10	Testing the Application	85
5.11	Security Considerations	86
5.12	Enhancements	86
5.13	Conclusion	86
	GDL/GWL Version of this Tutorial	89
	Bibliography	91
.1	Books	91
.1.1	Common Lisp: the Language, 2nd Edition	91
.1.2	ANSI Common Lisp	91
.1.3	Practical Common Lisp	91
.1.4	Paradigms of Artificial Intelligence Programming: Case Studies in Common Lisp	91
.1.5	Object-Oriented Programming in Common Lisp: A Programmer's Guide to CLOS	92
.1.6	On Lisp: Advanced Techniques for Common Lisp . . .	92
.1.7	Common Lisp Recipes	92
.2	Online Resources	92
.2.1	Common Lisp HyperSpec	92

.2.2	The Common Lisp Cookbook	92
.2.3	Planet Lisp	93
.2.4	Quicklisp	93
.2.5	The Association of Lisp Users	93
.3	Tutorials	93
.3.1	Lisp Primer from Tulane University	93
.3.2	Common Lisp: A Gentle Introduction to Symbolic Computation	93
.3.3	Successful Lisp	93
.4	Implementation-Specific Documentation	94
.4.1	Allegro CL Documentation	94
.4.2	SBCL Manual	94
.4.3	CCL Documentation	94
.5	Community and Discussion	94
.5.1	comp.lang.lisp	94
.5.2	Reddit /r/lisp	94
.5.3	Lisp Discord and IRC Channels	94
	Emacs Customization	95
.6	Key Bindings	95
.6.1	Remapping Caps Lock	95
.7	Essential Emacs Commands	96
.7.1	Navigation	96
.7.2	Editing	96
.7.3	Buffers and Windows	97
.7.4	Lisp-Specific Commands	97
.8	Customizing .emacs	98
.8.1	Basic .emacs Example	98
.8.2	Keyboard Macro for Common Lisp Session	99
.8.3	ParEdit Mode	99
.8.4	Rainbow Delimiters	99
.9	Package Management: MELPA	99
.10	SLIME: An Alternative to the Emacs-Lisp Interface	100
.11	Color Themes	100
.12	Other Useful Packages	101
.12.1	Company Mode	101
.12.2	Magit	101
.12.3	Projectile	101
.13	Learning More About Emacs	101

Afterword

103

Chapter 1

Introduction

1.1 The Past, Present, and Future of Common Lisp

1.1.1 Lisp Yesterday

John McCarthy discovered the basic principles of Lisp in 1958, when he was processing complex mathematical lists at MIT. Common Lisp (CL) is a high-level computer language, whose syntax follows a simple list-like structure. The term “Lisp” itself originally stood for “LIST Processing.” When developing, testing, and running a CL program, at the core is a modern-day version of the original List Processor which processes (compiles, evaluates, etc.) the elements of your program. These elements, at the source code level, are represented as lists. A list, in this context, is just a sequence of items, much like a familiar shopping list or checklist. Originally the list was pretty much the only data structure supported by Lisp, but modern-day Common Lisp supports a wide range of flexible and efficient data structures.

A typical Common Lisp development and runtime environment behaves like a complete operating system, with multiple threads of execution and the ability to load new code and redefine objects (functions, etc.) dynamically, i.e. without stopping and restarting the “machine.”

This “operating system” characteristic also makes CL significantly more flexible than other popular programming languages. Whereas some other languages, such as shell scripts or Perl CGI scripts, need to pipe data around, in CL all data manipulations can run interactively within one single process with a shared memory space.

1.1.2 Lisp Today

The most popular form of Lisp used today, “ANSI Common Lisp,” was initially designed by Guy Steele in *Common Lisp, the Language, 2nd Edition* (“CLtL2”) — (see the Bibliography in Appendix B). This version of Lisp became the accepted industry standard. In 1995, the American National Standards Institute recognized a slightly updated version as ANSI Common Lisp, the first object-oriented language to receive certification, and ANSI CL remains the only language that meets all of the criteria set forth by the Object Management Group (OMG) for a complete object-oriented language.

Paul Graham outlines ANSI CL in his 1996 book *ANSI Common Lisp*, also listed in Appendix B.

Official language standardization is important, because it protects developers from becoming saddled with legacy applications as new versions of a language are implemented. For example, this lack of standardization has been a continuing problem for Perl developers. Since each implementation of Perl defines the behavior of the language, it is not uncommon to have applications that require outdated versions of Perl, making it nearly impossible to use in a mission-critical commercial environment.

1.1.3 Lisp Tomorrow

Many developers once hoped that the software development process of the future would be more automated through Computer-aided Software Engineering (CASE) tools. Such tools claim to enable programmers and non-programmers to diagram their applications visually and automatically generate code. While useful to a certain extent, traditional CASE tools cannot cover domain-specific details and support all possible kinds of customizations — so developers inevitably need to hand-code the rest of their applications, breaking the link between the CASE model and the code. CASE systems fall short of being a true “problem-solving tool.”

The recursive nature of CL, and its natural ability to build applications in layers and build upon itself — in essence, code which writes code which writes code..., makes CL a much better software engineering solution. CL programs can generate other CL programs (usually at compile-time), allowing the developer to define all parts of the application in one unified high-level language. Using macros and functions, naturally supported by the CL syntax, the system can convert applications written in the high-level language automatically into “final” code. Thus, CL is both a programming language and a powerful CASE tool, and there is no need to break the connection.

Several other features of CL also save significant development time and manpower:

- **Automatic Memory Management/Garbage Collection:** inherent in CL's basic architecture.
- **Dynamic Typing:** In CL, values have types, but variables do not. This means that you don't have to declare variables, or placeholders, ahead of time to be of a particular type; you can simply create them or modify them on the fly¹.
- **Dynamic Redefinition:** You can add new operations and functionality to the running CL process without the need for downtime. In fact, a program can be running in one thread, while parts of the code are redefined in another thread. Moreover, as soon as the redefinitions are finished, the application will use the new code and is effectively modified while running. This feature is especially useful for server applications that cannot afford downtimes – you can patch and repair them while they are running.
- **Portability:** The “abstract machine” quality of CL makes programs especially portable.
- **Standard Features:** CL contains many built-in features and data structures that are non-standard in other languages. Features such as full-blown symbol tables and package systems, hash tables, list structures, and a comprehensive object system are several “automatic” features which would require significant developer time to recreate in other languages.

1.2 Convergence of Hardware and Software

The most common criticism of Lisp usually made by programmers unfamiliar with the language is that Lisp applications are too big and too slow. While this may have been true 20 years ago, it is absolutely not the case today. Today's inexpensive commodity computers have more memory than some of the most powerful machines available just five years ago, and easily meet the computing needs of a typical CL application. Hardware is no longer the critical factor. Programmers and development time are!

¹While you don't have to declare variable types, you may declare them. Doing so can help CL's compiler to optimize your program further.

Further, CL programming teams tend to be small, because the inherent features in CL empower developers to do more. Without excessive effort, a single CL developer can create and deploy a sophisticated, powerful program in a much shorter period of time than if another language were used.

1.3 The CL Model of Computation

Programming in CL is distinguished from programming in other languages due to its unique syntax and development mode. CL allows developers to make changes and test them immediately, in an incremental manner. In other languages, a developer must follow the “compile-link-run-test” cycle. These extra steps add minutes or hours to each cycle of development, and break the programmer’s thought process. Multiply these figures by days, weeks, and months — and the potential savings are phenomenal.

Chapter 2

Operating a CL Development Environment

This chapter covers some of the hands-on techniques used when working with a CL development environment, specifically, the Allegro CL[®] environment from Franz Inc. If you are completely unfamiliar with any Lisp language, you may wish to reference Chapter 3 as you read this chapter.

Ideally, while reading this chapter, you should have access to an actual Common Lisp session and try typing in some of the examples.

When working with Common Lisp, it helps to think of the environment as its own operating system, sitting on top of whatever operating system you happen to be working with. In fact, people used to build entire workstations and operating systems in Lisp. The Symbolics Common Lisp environment still runs as an emulated machine on top of the 64-bit Compaq/DEC Alpha CPU, with its own Lisp-based operating system.

A complete and current listing of available CL systems can be found on the Association of Lisp Users website, at

<http://www.alu.org/table/systems.htm>

These systems follow the same basic principles, and most of what you learn and write on one system can be applied and ported over to others. The examples in this guide were prepared using Allegro CL for Linux. Where possible, we will note Allegro CL-specific syntax and extensions.

2.1 Installing a CL Environment

Installing a CL environment is generally a straightforward process. For Allegro CL on Linux, this basically involves running a simple shell script which steps you through some licensing information, then proceeds to unpack the distribution, and sets up the CL executable and supporting files.

The Windows installation involves somewhat less typing and more mouse clicking.

2.2 Running CL in a Shell Window

The most primitive and simplest way to run CL is directly from the Unix (Linux) shell from within a terminal window. Most programmers do not work this way on a daily basis because other modes of use provide more power and convenience with fewer keystrokes. This mode of working is sometimes useful for logging into a running CL system on a remote host (e.g. a CL process acting as a webserver) when using a slow dial-up connection.

2.2.1 Starting CL from a Terminal Window

To start a CL session from a terminal window, simply type

```
alisp
```

from the Unix (or DOS) command line. Assuming your shell execution path is set up properly and CL is installed properly, you will see some introductory information, then will be presented with a Command Prompt which should look very similar to the following:

```
CL-USER(1):
```

CL has entered its read-eval-print loop, and is waiting for you to type something which it will then read, evaluate to obtain a return-value, and finally print this resulting return-value. The USER printed before the prompt refers to the current CL package (more on Packages later), and the number (1) simply keeps track of how many commands have been entered at the prompt.

2.2.2 Stopping CL from a Terminal Window

Now that you have learned how to start CL, you should probably know how to stop it. In Allegro CL, one easy way to stop the CL process is to type

```
(excl:exit)
```

at the Command Prompt. This should return you to the shell. In very rare cases, a stronger hammer is required to stop the CL process, in which case you can try typing

```
(excl:exit 0 :no-unwind t)
```

Try It: Now try starting and stopping CL several times, to get a feel for it. In general, you should notice that CL starts much more quickly on subsequent invocations. This is because the entire executable file has already been read into the computer's memory and does not have to be read from the disk every time.

A shortcut for the `(excl:exit)` command is the toplevel command `:exit`. Toplevel commands are words which are preceded by a colon `[:]` that you can type at the CL Command Prompt as a shorthand way of making CL do something.

Try It: Start the CL environment. Then, using your favorite text editor, create a file `/tmp/hello.lisp` and place the following text into it (don't worry about understanding the text for now):

```
(in-package :user)

(defun hello ()
  (write-string "Hello, World!"))
```

Save the file to disk. Now, at the CL prompt in the shell where you started the CL environment, compile and load this file as follows (text after the "USER" prompt represents what you have to type; everything else is text printed by CL):

```
CL-USER(3): (compile-file "/tmp/hello")
;;; Compiling file /tmp/hello.lisp
;;; Writing fasl file /tmp/hello.fasl Fasl write complete
#p"/tmp/hello.fasl"
NIL
```

```

NIL
CL-USER(4): (load "/tmp/hello")
; Fast loading /tmp/hello.fasl
T

```

By default, the `compile-file` command will look for files with the `.lisp` suffix, and the `load` command will load the resulting compiled machine-code (binary) file, which by default will have the extension `.fasl`. The extension “`.fasl`” stands for “FAST Loading” file.

Note that, in general, simply loading an uncompiled Lisp file (using `load`) will functionally have the same effect as loading a compiled binary (`fasl`) file. But the `fasl` file will load faster, and any functions, objects, etc. defined in it will perform faster, since they will have been optimized and translated into language which is closer to the actual machine. Unlike Java “class” files, CL `fasl` files usually represent native machine-specific code. This means that compiled CL programs will generally run much faster than compiled Java programs, but CL programs must be compiled separately for each type of machine on which you want to run them.

Finally, try executing the newly defined function `hello` by typing the following at your command line:

```

CL-USER(5): (hello)
Hello, World!
"Hello, World!"

```

You should see the String `Hello, World!` printed (without double-quotes), then the returned String (with double-quotes) will be printed as well. In the next chapter, we will learn more about exactly what is going on here. For now the main point to understand is the idea of compiling and loading definitions from files, and invoking functions by typing expressions at the CL command line.

2.3 Running CL inside a Text Editor

A more powerful way of developing with CL is by running it from within a Unix (or DOS) shell that is running inside a buffer in a powerful text editor, such as Gnu Emacs¹. One advantage of this approach is that the

¹Gnu Emacs comes pre-installed with all Linux systems, and for other systems should come as part of your Allegro CL distribution. In any case, Emacs distributions and documentation are available from <http://www.gnu.org>

editor (in our example, Emacs) keeps a history of the entire session: both the expressions the programmer has typed, as well as everything that CL has printed. All these items can be easily reviewed or recalled.

2.3.1 A Note on Emacs and Text Editors

To develop using Common Lisp, you can, of course, use any text editor of your choice, although Emacs happens to be especially synergistic for working with CL, for reasons we will discuss later. With a bit of practice, you will find that the combination of Emacs and Allegro CL provides a more powerful and convenient development environment than any of today's flashy "visual programming" environments. If you are used to working with the "vi" editor, Emacs comes with several "vi" emulation modes which you may want to experiment with on a transitional basis. If you are used to working with a standard Windows-based text editor, you will find a smooth transition to Emacs.

This guide provides examples using Gnu Emacs.

To work with CL in an Emacs shell buffer, you should be reasonably familiar with the text editor. If you are completely new to Gnu Emacs, go through the Emacs Tutorial, available under the "Help" menu at the top of the Emacs window.

2.3.2 Emacs Terminology

As you will learn in the Emacs Tutorial, when working with Emacs you make frequent use of key chords. Similar to "chords" on a piano, keychords can allow powerful text processing with relatively few sequential keystrokes.

They are key combinations involving holding down the Control key and/or the Meta key, and pressing some other regular key.

See Appendix C for information on which keys represent Control and Meta on your keyboard, and how you can customize these. To begin, try using the key labeled "Control" for Control and the keys labeled "Alt" (or the diamond-shaped key on Sun keyboards) for Meta. If your keyboard has a "Caps Lock" key to the left of the letter "A," you should consider remapping this to function as Control, as explained in Appendix C.

This guide uses the following notations to indicate keychords (identical to those referenced in the Emacs Tutorial):

- M-x means to hold down the Meta key, and press (in this case) the "X" key

- **C-x** means to hold down the Control key, and press (in this case) the “X” key
- **C-M-q** means to hold down the Control key and the Meta key at the same time , and press (in this case) the “Q” key
- **M-A** would mean to hold down the Meta key, and the Shift key (because the “A” is uppercase), and press (in this case) the “A” key.

2.3.3 Starting, Stopping, and Working With CL inside an Emacs Shell

To start Emacs, type

```
emacs
```

or

```
gnuemacs
```

then start a Unix (or DOS) shell within emacs by typing **M-x shell**. Now type

```
lisp
```

inside this shell’s window to start a CL session there. To shut down a session, exit CL exactly as above by typing

```
(excl:exit)
```

Exit from the shell by typing

```
exit
```

and finally exit from Emacs by typing **C-x C-c** or **M-x kill-emacs**. Note that it is good practice always to exit from CL before exiting from Emacs. Otherwise the CL process may be left as an “undead” (zombie) process.

When in the CL process, you can move forward and backward in the “stack” (history) of expressions that you have typed, effectively recalling previous commands with only a few keystrokes. Use **M-p** to move backward in the history, and **M-n** to move forward. You can also use all of the editor’s text processing commands, for example to cut, copy, and paste expressions

between the command line and other windows and open files (“buffers”) you may have open.

Try It: try typing the following expressions at the CL prompt. See what return-values are printed, and try recalling previous expressions (commands) using M-p and M-n:

```
(list 1 2 3)
```

```
(+ 1 2 3)
```

```
(> 3 4)
```

```
(< 3 4)
```

Now, of course, you can edit, compile, load, and test your `hello.lisp` file as we did in Section 2.2. But this time you are staying within a single environment (the text editor environment) to achieve all these tasks.

2.4 Running CL as a subprocess of Emacs

An even more powerful way of developing with Allegro CL is by running it within Emacs in conjunction with a special Emacs/Lisp interface provided by Franz Inc. This technique provides all the benefits of running in an Emacs shell as outlined above, but is more powerful because the Emacs-CL interface extends Emacs with several special commands. These commands interact with the CL process, making the editor appear to be “Lisp-aware.” The combination of Emacs with the Emacs-CL interface results in a development environment whose utility approaches that of the vintage Symbolics Lisp Machines, which many Lisp aficionados still consider to be technology advanced well beyond anything produced today.

Complete documentation of the Emacs-CL interface can be viewed by pointing your web browser at: `file:/usr/local/acl62/doc/eli.htm` (you may have to modify this pathname to point to your actual installed location of Allegro CL (acl62)).

2.4.1 Starting the CL subprocess within Emacs

To enable your Emacs to function in the proper manner, place the following expressions in a file named `.emacs` in your home directory:

```
(setq load-path
      (cons "/usr/local/acl62/eli" load-path))
(load "fi-site-init")
```

As above, you may have to modify this pathname to point to your actual installed location of Allegro CL (acl62). Once you have added these lines to your `.emacs` file, you should be able to restart emacs, then issue the command:

```
M-x fi:common-lisp
```

to start CL running and automatically establish a network connection between the Emacs process and the CL process (in theory, the two processes could be running on different machines, but in practice they usually will run on the same machine). Accept all the defaults when prompted.

You can also start emacs with the CL subprocess automatically from a terminal window, by invoking emacs with an optional “function” argument, as follows:

```
emacs -f fi:common-lisp
```

This will start Emacs, which itself will immediately launch CL.

2.4.2 Working with CL as an Emacs subprocess

Once you have started CL within Emacs, you will be presented with a special buffer named `*common-lisp*` which in many ways is similar to simply running CL within a shell within Emacs as above. But now you have several more capabilities available to you, as outlined in Table 2.1.

To summarize, the Lisp-Emacs interface adds capabilities such as the following to your development environment:

- Start and stop the CL process using Emacs commands
- Compile and load files into the running CL process, directly from the open Emacs buffer you are editing (no need explicitly to save the file, call the `compile-file` function, then call the `load` function, as with the “in-a-shell” techniques above)
- Query the running CL process for information about objects currently defined in it — for example, querying the CL, directly from Emacs, for the argument List of a particular function

Keychord	Action	Emacs-Lisp function
C-M-x	Compiles and loads the function near the Emacs point	fi:lisp-eval-or-compile-defun
C-c C-b	Compiles and loads the current Emacs buffer	fi:lisp-eval-or-compile-current-buffer
M-A	Shows the Argument List for a CL function	fi:lisp-arglist
C-c .	Finds the definition of a CL object	fi:lisp-find-definition

Table 2.1: Common Commands of the Emacs-Lisp Interface

- Run multiple read-eval-print loops, or listeners, in different threads of CL execution, as Emacs buffers
- Compile and load specified sections of an open Emacs buffer
- Locate the source code file corresponding to any defined CL object, such as a function or parameter, and automatically open it in an Emacs buffer
- Perform various debugging actions and other general CL help capabilities, directly from within the Emacs session

In short, Emacs becomes a remarkably powerful, infinitely customizable, CL integrated development environment.

2.4.3 Compiling and Loading a File from an Emacs buffer

Try it: Try starting Emacs with a CL subprocess as outlined above. If you have trouble, consult the URL listed above for complete instructions on running the Lisp-Emacs interface.

Once you have Emacs running with a CL subprocess, you should have a buffer named `*common-lisp*`. If your Emacs session is not already visiting this buffer, visit it with the command `C-x b *common-lisp*`, or select `*common-lisp*` from the “Buffers” menu in Emacs (see Appendix C for instructions on how to set up a “hot key” for this and other common tasks when working with the Emacs-Lisp interface).

Now, split your Emacs frame into two windows, with the command `C-x 2`. In both windows you should now see `*common-lisp*`. Now move the point² to the other window with `C-x o`, and open the `hello.lisp` file you created for the previous exercise³. If you don't have that file handy, create it now, with the following contents:

```
(defun hello ()
  (write-string "Hello, World!"))
```

Now, compile and load the contents of this function definition with `C-M-x`. Finally, switch to the `*common-lisp*` window again with `C-x o`, and try invoking the `hello` function by calling it in normal Lisp fashion:

```
CL-USER(5): (hello)
Hello, World!
"Hello, World!"
```

You should see the results printed into the same `*common-lisp*` buffer.

Try it: Now try the following: go back to the `hello.lisp` buffer, make an edit to the contents (for example, change the String to “Hello, CL World!”). Now compile the buffer with this change (Note that you do not necessarily have to save the file). Finally, return to the `*common-lisp*` buffer, and try invoking the function again to confirm that the redefinition has taken effect.

You have now learned the basic essentials for working with the Emacs-Lisp interface. See Appendix C for more information about Emacs customization and convenient Emacs commands specific to the Lisp editing mode.

2.5 Integrated Development Environment

Some CL implementations support or integrate with an IDE, or Integrated Development Environment. Such environments provide visual layout and design capabilities for User Interfaces and other graphical, visual techniques for application development. For example, such a system is available for Allegro CL on the Microsoft platforms. The details of these systems are covered in their own documentation and are beyond the scope of this guide.

²In technical Emacs parlance, the point is the insertion point for text, and the cursor is the mouse pointer.

³Remember that pressing the space bar will automatically complete filenames for you when opening files in Emacs.

2.6 The User Init File

When CL begins execution, it usually looks for an initialization file, or “init” file, to load. When CL loads this file, it will read and evaluate all the commands contained in it.

The default name and location for this initialization file is

```
.clinit.cl
```

in the user’s home directory on Unix/Linux, and

```
c:\clinit.cl
```

on Windows systems.

For example, the init file can be used to define library functions and variables that the user employs frequently, or to control other aspects of the CL environment. An example init file is:

```
(in-package :user)

(defun load-project ()
  (load "~/lisp/part1")
  (load "~/lisp/part2"))
```

This will have the effect of defining a convenient function the user can then invoke to load the files for the current project. Note the call to the `in-package` function, which tells CL which Package to make “current” when loading this file. As we will see in more detail in Section 3.7, CL Packages allow programs to be separated into different “layers,” or “modules.” Notice also that in the calls to `load` we do not specify a file “type” extension, e.g. `.lisp` or `.fasl`. We are making use of the fact that the `load` function will look first for a binary file (of type `fasl`), and if this is not found it will then look for a file of type `cl` or `lisp`. In this example we assume that our Lisp files will already have been compiled into up-to-date binary `fasl` files.

In Section 2.9 we will look at some techniques for managing larger projects, and automating both the compiling and loading process for the source files making up a project.

Note: the IDE mentioned in Section 2.5 is also a full-featured and powerful project system. The IDE is described in its own documentation, and is beyond the scope of this document.

2.7 Using CL as a scripting language

Although CL is usually used in the interactive mode described above while developing a program, it can be also used as a scripting language in more of a “batch” mode. For example, by running CL as:

```
lisp -e '(load "~/script")' < datafile1 > datafile2
```

it will load all of the function definitions and other expressions in file `script.fasl` or `script.lisp`, will process `datafile1` on its Standard Input, and will create `datafile2` with anything it writes to its Standard Output. Here is an example of a simple `script.lisp` file:

```
(in-package :user)

(defun process-data ()
  (let (x j)
    ;;Read a line of input from datafile1
    (setq x (read-line))

    ;;Remove leading spaces from X
    (setq j (dotimes (i (length x))
      (when (not (string-equal " " (subseq x i (+ i 1))))
        (return i))))
    (setq x (subseq x j))

    ;;Write the result to datafile2
    (format t "~A~%" x)))

(process-data)
```

This file defines a function to process the datafile, then calls this function to cause the actual processing to occur.

Note the use of the semicolon (“;”), which is a reader macro which tells CL to treat everything between it and the end of the line as a comment (to be ignored by the CL Reader).

2.8 Debugging in CL

CL provides one of the most advanced debugging and error-handling capabilities of any language or environment. The most obvious consequence of this

is that your application will very rarely “crash” in a fatal manner. When an error occurs in a CL program, a break occurs. CL prints an error message, enters the debugger, and presents the user with one or more possible restart actions. This is similar to running inside a debugger, or in “debug mode,” in other languages, but in CL this ability comes as a built-in part of the language.

For example, if we call the `+` function with an alphabetic Symbol instead of a Number, we generate the following error:

```
CL-USER(95): (+ 'r 4)
Error: 'R' is not of the expected type 'NUMBER'
[condition type: TYPE-ERROR]

Restart actions (select using :continue):
0: Return to Top Level (an "abort" restart)
[1] CL-USER(96):
```

The number `[1]` at the front of the prompt indicates that we are in the debugger, at Level 1. The debugger is itself a CL Command Prompt just like the toplevel Command Prompt, but it has some additional functionality. The next section provides a partial list of commands available to be entered directly at the debugger’s Command Prompt:

2.8.1 Common debugger commands

- `:reset` returns to the toplevel (out of any debug levels)
- `:pop` returns up one level
- `:continue` continues execution after a break or an error
- `:zoom` Views the current function call stack⁴
- `:up` Moves the currency pointer up one frame in the stack
- `:down` Moves the currency pointer down one frame in the stack
- `:help` Describes all debugger commands

Franz Inc.’s Allegro Composer product also offers graphical access to these commands, as well as additional functionality.

⁴the call stack is the sequence of functions calls leading up to the error or break.

2.8.2 Interpreted vs Compiled Code

CL works with both compiled and interpreted code. When you type expressions at a Command Prompt, or use CL's `load` function to load lisp source files directly, you are introducing interpreted code into the system. This code does not undergo any optimization or translation into a lower-level machine representation — internally to CL it exists in a form very similar to its source code form. CL must do considerable work every time such code is evaluated, since it must be translated into lower-level machine instructions on each use. When you create compiled code, on the other hand, for example by using CL's `compile-file` command in order to produce a fasl file, the code is optimized and translated into an efficient form. Loading the fasl file will replace any previous interpreted definitions with compiled definitions.

Interpreted code is the best to use for debugging a program, because the code has not undergone as many transformations, so the debugger can provide the programmer with more recognizable information.

If you are debugging with compiled code (e.g. after loading a fasl file), and you feel that not enough information is available from within the debugger, try redefining the function in interpreted form and running it again. To redefine a function in interpreted form you can, of course, simply load the lisp source file, rather than the compiled fasl file.

2.8.3 Use of (break) and C-c to interrupt

In order to cause a break in your program interactively, issue the `C-c` command. When running inside Emacs, you must use a “double” `C-c`, because the first `C-c` will be intercepted by Emacs. Depending on what your application is doing, it may take a few moments for it to respond to your break command. Once it responds, you will be given a normal debugger level, as described above.

2.8.4 Profiling

CL has built-in functionality for monitoring its own performance. The most basic and commonly used component is the `time` Macro. You can “wrap” the `time` Macro around any expression. Functionally, `time` will not affect the behavior of the expression, but it will print out information about how long the expression took to be evaluated:

```
CL-USER(2): (time (dotimes (n 10000) (* n 2)))
; cpu time (non-gc)      70 msec user, 0 msec system
```

```

; cpu time (gc)           0 msec user, 0 msec system
; cpu time (total)       70 msec user, 0 msec system
; real time              72 msec

```

It is often interesting to look at the difference in time between interpreted and compiled code. The example above is interpreted, since we typed it directly at the Command Line, so CL had no opportunity to optimize the dotimes. In compiled form, the above code consumes only one millisecond as opposed to 72 milliseconds.

In addition to the basic `time` Macro, Allegro CL provides a complete Profiler package, which allows you to monitor exactly what your program is doing, when it is doing it, how much time it takes, and how much memory it uses.

2.9 Developing Programs and Applications in CL

For developing a serious application in CL, you will want to have a convenient procedure in place for starting up your development session.

2.9.1 A Layered Approach

The development session consists of your favorite text editor with appropriate customizations, and a running CL process with any standard code loaded into it. While you can certainly develop serious applications on top of base CL, typically you will work in a layered fashion, on top of some standard tools or libraries, such as an embedded language, a webserver, etc.

Using one of the techniques outlined below, starting a customized development session with your standard desired tools should become convenient and transparent.

2.9.2 Compiling and Loading your Project

An individual project will generally consist of a tree of directories and files in your computer's filesystem. We will refer to this directory tree as the project's codebase.

Usually, it does not matter in what order the files in the codebase are compiled and loaded. Typical CL definitions, such as functions, classes, etc., can ordinarily be defined in any order. However, certain constructs, most notably Packages and Macros, do have order dependencies. In general, CL Packages and Macros must be defined before they can be used, or referenced.

To start a development session on your project, you will generally want CL to do the following:

1. Go through your project's Codebase, finding source files (in the proper order when necessary, as noted above)
2. For each file found, either load the corresponding binary file (if it is up-to-date), or compile the source file to create a new binary file and load it (if the source file has changed since the most recent compilation)

CL does not dictate one standard way of accomplishing this task, and in fact, several options are available. One option is to use a defsystem package, which will allow you to prepare a special file, similar to a “make” file, which lists out all your project's source files and their required load order. Allegro CL contains its own Defsystem package as an extension to CL, and the open-source MK:Defsystem is available through Sourceforge (<http://www.sourceforge.net>).

Using a Defsystem package requires you to maintain a catalog listing of all your project's source files. Especially in the early stages of a project, when you are adding, renaming, and deleting a lot of files and directories, maintaining this catalog listing by hand can become tiresome.

For this purpose, you may wish to use a lighter-weight utility for compiling and loading your files, at least in the early stages of development. For very small projects, you can use the Allegro CL function `excl:compile-file-if-needed`, an extension to CL, by writing a simple function which calls this function repeatedly to compile and load your project's files.

The Bootstrap package, available at <http://gdl.sourceforge.net>, provides the function `cl-lite`, which will traverse a codebase, compiling and loading files, and observing simple “ordering directive” files which can be placed throughout the codebase to enforce correct load ordering.

For a new project, the best option is probably to start with a simple but somewhat manual loading technique, then graduate into more automated techniques as they become necessary and as you become more familiar with the environment.

2.9.3 Creating an Application “Fasl” File

Fasl files in Allegro CL and most other CL systems have the convenient property that they can simply be concatenated, e.g. with the Unix/Linux `cat` command, to produce a single fasl file. Loading this single file will have

the same effect as loading all the individual files which went into it, in the same order as they were concatenated.

In certain situations, this can be a convenient mechanism for loading and/or distributing your project. For example, you can prepare a concatenated fasl file, email it to a colleague, and the colleague needs only to load this file. Thus, there is no chance of confusion about how (e.g. in what order) to load individual files.

2.9.4 Creating an Image File

Loading Lisp source files or compiled binary files into memory at the start of each session can become slow and awkward for large programs.

An alternative approach is to build an image file. This entails starting a CL process, loading all application files into the running CL process, then creating a new “CL Image.” In Allegro CL, this is done by invoking the function:

```
excl:dumplisp
```

The Image File is typically named with a `.dxl` extension. Once you have created an Image File, you can start a session simply by starting CL with that Image File, as follows:

```
lisp -I <image-file-name>.dxl
```

You must then compile/load any files that have changed since the Image File was created. Starting CL with the Image File has the same effect as starting a base CL and loading all your application files, but is simpler and much faster.

2.9.5 Building Runtime Images

CL Image files built with `excl:dumplisp` contain the entire CL environment, and may also contain information which is specific to a certain machine or certain site. Allegro CL also provides a mechanism for building runtime images, which contain only the information needed to run an end-user application. These Runtime Images can be packaged and distributed just as with any stand-alone software application.

Preparing runtime images for different platforms (e.g. Sun Solaris, Linux, Windows, etc.) is usually a simple matter of running a compilation and building a runtime image for the target platform — application source code typically remains identical among different platforms. See your platform-specific documentation for details about creating Runtime Images.

2.9.6 Using an Application Init File

In some cases, for a finished production application, you will want the CL process to execute certain commands upon startup (for example, loading some data from a database). However, you do not want to depend on the end-user having the appropriate `.clinit.cl` in his or her home directory. For such cases, you can specify a different location for the init file, which can be installed along with the application itself. One way to accomplish this is with the `-q` command-line argument to the `lisp` command. This argument will tell CL to look in the current directory, rather than in the default location, for the init file.

To use this technique, you would set up an “application home” directory, place the application’s `.clinit.cl` file in this directory, and start the application with a production startup script. This script would first change into the “application home” directory, then start CL with your application’s image file, or start a Runtime Image. Such a startup script might look like this:

```
cd $MYAPP_HOME
lisp -I image.dxl -q
```

In this example, both the application’s init file and the application’s image file would be housed in the directory named by the environment variable `$MYAPP_HOME`.

Chapter 3

The CL Language

If you are new to the CL language, we recommend that you supplement this chapter with other resources. See the Bibliography in Appendix B for some suggestions. The Bibliography also lists two interactive online CL tutorials from Tulane University and Texas A&M, which you may wish to visit.

In the meantime, this chapter will provide a condensed overview of the language. Please note, however, that this book is intended as a summary, and will not delve into some of the more subtle and powerful techniques possible with Common Lisp.

3.1 Overview of CL and its Syntax

The first thing you should observe about CL (and most languages in the Lisp family) is that it uses a generalized prefix notation.

One of the most frequent actions in a CL program, or at the toplevel read-eval-print loop, is to call a function. This is most often done by writing an expression which names the function, followed by its arguments. Here is an example:

```
(+ 2 2)
```

This expression consists of the function named by the symbol “+,” followed by the arguments 2 and another 2. As you may have guessed, when this expression is evaluated it will return the value 4.

Try it: Try typing this expression at your command prompt, and see the return-value being printed on the console.

What is it that is actually happening here? When CL is asked to evaluate an expression (as in the toplevel read-eval-print loop), it evaluates the expression according to the following rules:

1. If the expression is a number (i.e. looks like a number), it simply evaluates to itself (a number):

```
CL-USER(8): 99
99
```

2. If the expression looks like a string (i.e. is surrounded by double-quotes), it also simply evaluates to itself:

```
CL-USER(9): "Be braver -- you can't cross a chasm in two small jumps."
"Be braver -- you can't cross a chasm in two small jumps."
```

3. If the expression looks like a literal symbol, it will simply evaluate to that symbol (more on this in Section 3.1.4):

```
CL-USER(12): 'my-symbol
MY-SYMBOL
```

4. If the expression looks like a list (i.e. is surrounded by parentheses), CL assumes that the first element in this list is a symbol which names a function or a macro, and the rest of the elements in the list represent the arguments to the function or macro. (We will discuss functions first, macros later). A function can take zero or more arguments, and can return zero or more return-values. Often a function only returns one return-value:

```
CL-USER(14): (expt 2 5)
32
```

Try it: Try typing the following functional expressions at your command prompt, and convince yourself that the printed return-values make sense:

```
(+ 2 2)
```

```
(+ 2)
```

2

(+)

(+ (+ 2 2) (+ 3 3))

(+ (+ 2 2))

(sys:user-name)

(user-homedir-pathname)

(get-universal-time) ;; returns number of seconds since January 1, 1900

(search "Dr." "I am Dr. Strangelove")

"I am Dr. Strangelove"

(subseq "I am Dr. Strangelove"
 (search "Dr." "I am Dr. Strangelove"))

3.1.1 Evaluation of Arguments to a Function

Note that the arguments to a function can themselves be any kind of the above expressions. They are evaluated in order, from left to right, and finally they are passed to the function for it to evaluate. This kind of nesting can be arbitrarily deep. Do not be concerned about getting lost in an ocean of parentheses — most serious text editors can handle deeply nested parentheses with ease, and moreover will automatically indent your expressions so that you, as the programmer, never have to concern yourself with matching parentheses.

3.1.2 Lisp Syntax Simplicity

One of the nicest things about Lisp is the simplicity and consistency of its syntax. What we have covered so far pertaining to the evaluation of arguments to functions is just about everything you need to know about the syntax. The rules for macros are very similar, with the primary difference being that all the arguments of a macro are not necessarily evaluated at the

time the macro is called — they can be transformed into something else first.

This simple consistent syntax is a welcome relief from other widely used languages such as C, C++, Java, Perl, and Python. These languages claim to use a “more natural” “infix” syntax, but in reality their syntax is a confused mixture of prefix, infix, and postfix. They use infix only for the simplest arithmetic operations such as

`2 + 2`

and

`3 * 13`

and use postfix in certain cases such as

`i++`

and

`char*`

But mostly they actually use a prefix syntax much the same as Lisp, as in:

`split(@array, ":");`

So, if you have been programming in these other languages, you have been using prefix notation all along, but may not have noticed it. If you look at it this way, the prefix notation of Lisp will seem much less alien to you.

3.1.3 Turning Off Evaluation

Sometimes you want to specify an expression at the toplevel read-eval-print loop, or inside a program, but you do not want CL to evaluate this expression. You want just the literal expression. CL provides the special operator `quote` for this purpose. For example,

`(quote a)`

will return the literal symbol `A`. Note that, by default, the CL reader will convert all symbol names to uppercase at the time the symbol is read into the system. Note also that symbols, if evaluated “as is,” (i.e. without the quote), will by default behave as variables, and CL will attempt to retrieve an associated value. More on this later.

Here is another example of using `quote` to return a literal expression:

```
(quote (+ 1 2))
```

Unlike evaluating a list expression “as is,” this will return the literal list `(+ 1 2)`. This list may now be accessed as a normal list data structure¹.

Common Lisp defines the abbreviation `'` as a shorthand way to “wrap” an expression in a call to `quote`. So the previous examples could equivalently be written as:

```
'a
```

```
'(+ 1 2)
```

3.1.4 Fundamental CL Data Types

Common Lisp natively supports many data types common to other languages, such as numbers, strings, and arrays. Native to CL is also a set of types which you may not have come across in other languages, such as lists, symbols, and hash tables. In this overview we will touch on numbers, strings, symbols, and lists. Later in the book we will provide more detail on some CL-specific data types.

Regarding data types, CL follows a paradigm called dynamic typing. Basically this means that values have type, but variables do not necessarily have type, and typically variables are not “pre-declared” to be of a particular type.

Numbers

Numbers in CL form a hierarchy of types, which includes Integers, Ratios, Floating Point, and Complex numbers. For many purposes you only need to think of a value as a “number,” without getting any more specific than that. Most arithmetic operations, such as `+`, `-`, `*`, `/`, etc, will automatically do any

¹It should be noted, however, that when a quoted literal list is created like this, it is now technically a constant — you should not modify it with destructive operators.

necessary type coercion on their arguments and will return a number of the appropriate type.

CL supports a full range of floating-point decimal numbers, as well as true Ratios, which means that $1/3$ is a true one-third, not 0.33333333 rounded off at some arbitrary precision.

As we have seen, numbers in CL are a native data type which simply evaluate to themselves when entered at the toplevel or included in an expression.

Strings

Strings are actually a specialized kind of Array, namely a one-dimensional array (vector) made up of characters. These characters can be letters, numbers, or punctuation, and in some cases can include characters from international character sets (e.g. Unicode) such as Chinese Hanzi or Japanese Kanji. The string delimiter in CL is the double-quote character (“”).

As we have seen, strings in CL are a native data type which simply evaluate to themselves when included in an expression.

Symbols

Symbols are such an important data structure in CL that people sometimes refer to CL as a “Symbolic Computing Language.” Symbols are a type of CL object which provides your program with a built-in mechanism to store and retrieve values and functions, as well as being useful in their own right. A symbol is most often known by its name (actually a string), but in fact there is much more to a symbol than its name. In addition to the name, symbols also contain a function slot, a value slot, and an open-ended Property-list slot in which you can store an arbitrary number of named properties.

For a named function such as `+`, the function-slot of the symbol `+` contains the actual function itself. The value-slot of a symbol can contain any value, allowing the symbol to act as a global variable, or Parameter. And the Property-list, or plist slot, can contain an arbitrary amount of information.

This separation of the symbol data structure into function, value, and plist slots is one obvious distinction between Common Lisp and most other Lisp dialects. Most other dialects allow only one (1) “thing” to be stored in the symbol data structure, other than its name (e.g. either a function or a value, but not both at the same time). Because Common Lisp does not impose this restriction, it is not necessary to contrive names, for example for your variables, to avoid conflicting with existing “reserved words” in the

system. For example, “list” is the name of a built-in function in CL. But you may freely use “list” as a variable as well. There is no need to contrive arbitrary abbreviations such as “lst.”

How symbols are evaluated depends on where they occur in an expression. As we have seen, if a symbol appears first in a list expression, as with the + in (+ 2 2), the symbol is evaluated for its function slot. If the first element of an expression is a symbol which indeed does contain a function in its function slot, then any symbol which appears anywhere else (i.e. in the rest) of the expression is taken as a variable, and it is evaluated for its global or local value, depending on its scope. More on variables and scope later.

As noted in Section 3.1.3, if you want a literal symbol itself, one way to achieve this is to “quote” the symbol name:

```
'a
```

Another way is for the symbol to appear within a quoted list expression:

```
'(a b c)
```

```
'(a (b c) d)
```

Note that the quote (') applies across everything in the list expression, including any subexpressions.

Lists

Lisp takes its name from its strong support for the list data structure. The list concept is important to CL for more than this reason alone — lists are important because all CL programs themselves are lists! Having the list as a native data structure, as well as the form of all programs, means that it is especially straightforward for CL programs to compute and generate other CL programs. Likewise, CL programs can read and manipulate other CL programs in a natural manner. This cannot be said of most other languages, and is one of the primary distinguishing characteristics of CL.

Textually, a list is defined as zero or more elements surrounded by parentheses. The elements can be objects of any valid CL data types, such as numbers, strings, symbols, lists, or other kinds of objects. As we have seen, you must quote a literal list to evaluate it or CL will assume you are calling a function.

Now look at the following list:

```
(defun hello () (write-string "Hello, World!"))
```

This list also happens to be a valid CL program (function definition, in this case). Don't worry about analyzing the function right now, but do take a few moments to convince yourself that it meets the requirements for a list. What are the types of the elements in this list?

In addition to using the quote (') to produce a literal list, another way to produce a list is to call the function `list`. The function `list` takes any number of arguments, and returns a list made up from the result of evaluating each argument (as with all functions, the arguments to the `list` function get evaluated, from left to right, before being passed into the function). For example,

```
(list 'a 'b (+ 2 2))
```

will return the list (A B 4). The two quoted symbols evaluate to symbols, and the function call (+ 2 2) evaluates to the number 4.

3.1.5 Functions

Functions form the basic building block of CL. Here we will give a brief overview of how to define a function; later we will go into more detail on what a function actually is.

A common way to define named functions in CL is with the macro `defun`, which stands for DEFinition of a FUNction. `Defun` takes as its arguments a symbol, an argument list, and a body:

```
(defun my-first-lisp-function ()
  (list 'hello 'world))
```

Because `defun` is a macro, rather than a function, it does not follow the hard-and-fast rule that all its arguments are evaluated as expressions — specifically, the symbol which names the function does not have to be quoted, nor does the argument list. These are taken as a literal symbol and a literal list, respectively.

Once the function has been defined with `defun`, you can call it just as you would call any other function, by wrapping it in parentheses together with its arguments:

```
CL-USER(56): (my-first-lisp-function)
(HELLO WORLD)
```

```
CL-USER(57): (defun square(x)
              (* x x))
SQUARE
CL-USER(58): (square 4)
16
```

Declaring the types of the arguments to a function is not required.

3.1.6 Global and Local Variables

Global Variables

Global variables in CL are usually also known as special variables. They can be established by calling `defparameter` or `defvar` from the read-eval-print loop, or from within a file that you compile and load into CL:

```
(defparameter *todays-temp* 90)

(defvar *todays-humidity* 70)
```

The surrounding asterisks on these symbol names are part of the symbol names themselves, and have no significance to CL. This is simply a long-standing naming convention for global variables (i.e. parameters), to make them easy to pick out with the human eye.

`Defvar` and `defparameter` differ in one important way: if `defvar` is evaluated with a symbol which already has a global value, it will not overwrite it with a new value. `Defparameter`, on the other hand, will overwrite it with a new value:

```
CL-USER(4): *todays-temp*
90
CL-USER(5): *todays-humidity*
70
CL-USER(6): (defparameter *todays-temp* 100)
*TODAYS-TEMP*
CL-USER(7): (defvar *todays-humidity* 50)
*TODAYS-HUMIDITY*
CL-USER(8): *todays-temp*
100
CL-USER(9): *todays-humidity*
70
```

`*todays-humidity*` did not change because we used `defvar`, and it already had a previous value.

The value for any Parameter can always be changed from the toplevel with `setq`:

```
CL-USER(11): (setq *todays-humidity* 30)
30
CL-USER(12): *todays-humidity*
30
```

Although `setq` will work with new variables, stylistically it should only be used with already established variables.

During application development, it often makes sense to use `defvar` rather than `defparameter` for variables whose values might change during the running and testing of the program. This way, you will not unintentionally reset the value of a parameter if you compile and load the source file where it is defined.

Local Variables

New local variables can be introduced with the macro `let`:

```
CL-USER(13): (let ((a 20)
                  (b 30))
              (+ a b))
50
```

As seen in the above example, `let` takes an assignment section and a body. `let` is a macro, rather than a function², so it does not follow the hard-and-fast rule that all its arguments are evaluated. Specifically, the assignment section

```
((a 20)
 (b 30))
```

is not evaluated (actually, the macro has the effect of transforming this into something else before the CL evaluator even sees it). Because this assignment section is not evaluated by the `let` macro, it does not have to be quoted, like a list which is an argument to a function would have to be. As you can see, the assignment section is a list of lists, where each internal

²See Section 3.8.5 for a discussion of how to recognize macros and functions

list is a pair whose first is a symbol, and whose second is a value (which will be evaluated). These symbols (**a** and **b**) are the local variables, and they are assigned to the respective values.

The body consists of any number of expressions which come after the assignment section and before the closing parenthesis of the **let** statement. The expressions in this body are evaluated normally, and of course any expression can refer to the value of any of the local variables simply by referring directly to its symbol. If the body consists of more than one expression, the final return-value of the body is the return-value of its last expression.

New Local variables in CL are said to have lexical scope, which means that they are only accessible in the code which is textually contained within the body of the **let**. The term lexical is derived from the fact that the behavior of these variables can be determined simply by reading the text of the source code, and is not affected by what happens during the program's execution.

Dynamic scope happens when you basically mix the concept of global parameters and local **let** variables. That is, if you use the name of a previously established parameter inside the assignment section of a **let**, like this:

```
(let ((*todays-humidity* 50))  
  (do-something))
```

the parameter will have the specified value not only textually within the the body of the **let**, but also in any functions which may get called from within the body of the **let**. The global value of the parameter in any other contexts will not be affected. Because this scoping behavior is determined by the runtime “dynamic” execution of the program, we refer to it as dynamic scope.

Dynamic scope is often used for changing the value of a particular global parameter only within a particular tree of function calls. Using Dynamic scope, you can accomplish this without affecting other places in the program which may be referring to the parameter. Also, you do not have to remember to have your code “reset” the parameter back to a default global value, since it will automatically “bounce” back to its normal global value.

Dynamic scoping capability is especially useful in a multithreaded CL, i.e., a CL process which can have many (virtually) simultaneous threads of execution. A parameter can take on a dynamically scoped value in one thread, without affecting the value of the parameter in any of the other concurrently running threads.

3.2 The List as a Data Structure

In this section we will present some of the fundamental native CL operators for manipulating lists as data structures. These include operators for doing things such as:

1. finding the length of a list
2. accessing particular members of a list
3. appending multiple lists together to make a new list
4. extracting elements from a list to make a new list

3.2.1 Accessing the Elements of a List

Common Lisp defines the accessor functions `first` through `tenth` as a means of accessing the first ten elements in a list:

```
CL-USER(5): (first '(a b c))  
A  
CL-USER(6): (second '(a b c))  
B  
CL-USER(7): (third '(a b c))  
C
```

For accessing elements in an arbitrary position in the list, you can use the function `nth`, which takes an integer and a list as its two arguments:

```
CL-USER(8): (nth 0 '(a b c))  
A  
CL-USER(9): (nth 1 '(a b c))  
B  
CL-USER(10): (nth 2 '(a b c))  
C  
CL-USER(11): (nth 12 '(a b c d e f g h i j k l m n o p))  
M
```

Note that `nth` starts its indexing at zero (0), so `(nth 0 ...)` is equivalent to `(first ...)`, and `(nth 1 ...)` is equivalent to `(second ...)`, etc.

3.2.2 The “Rest” of the Story

A very common operation in CL is to perform some operation on the first of a list, then perform the same operation on each first of the rest of the list, repeating the procedure until the end of the list, i.e. the Empty list, is reached. The function `rest` is very helpful in such cases:

```
CL-USER(59): (rest '(a b c))  
(B C)
```

As you can see from this example, `rest` returns a list consisting of all but the first of its argument.

3.2.3 The Empty List

The symbol `NIL` is defined by Common Lisp to be equivalent to the Empty List, `()`. `NIL` also has the interesting property that its value is itself, which means that it will always evaluate to itself, whether or not it is quoted. So `NIL`, `'NIL`, and `()` all evaluate to the Empty List, whose default printed representation is `NIL`:

```
CL-USER(14): nil  
NIL  
CL-USER(15): 'nil  
NIL  
CL-USER(16): ()  
NIL
```

The function `null` can be used to test whether or not something is the Empty List:

```
CL-USER(17): (null '(a b c))  
NIL  
CL-USER(18): (null nil)  
T  
CL-USER(19): (null ())  
T
```

As you may have deduced from the above examples, the symbol `T` is CL's default representation for “true.” As with `NIL`, `T` evaluates to itself.

3.2.4 Are You a List?

To test whether or not a particular object is a list, CL provides the function `listp`. Like many other functions which end with the letter “p,” this function takes a single argument and checks whether it meets certain criteria (in this case, whether it qualifies as a list). These predicate functions ending in the letter “p” will always return either T or NIL:

```
CL-USER(20): (listp '(pontiac cadillac chevrolet))
T
CL-USER(21): (listp 99)
NIL
CL-USER(22): (listp nil)
T
```

Note that `(listp nil)` returns T, since NIL is indeed a list (albeit the empty one).

3.2.5 The conditional If

Before continuing with a number of other basic list functions, we will cover the macro `if`, which allows simple conditionals. `If` takes three arguments, a test-form, a then-form, and an else-form. When an `if` form is evaluated, it first evaluates its test-form. If the form returns non-NIL, it will evaluate the then-form, else it will evaluate the else-form. The nesting of multiple `if` expressions is possible, but not advised; later we will cover other constructs which are more appropriate for such cases.

Here are some simple examples using the `if` operator:

```
CL-USER(2): (if (> 3 4)
                "yes"
                "no")
"no"
CL-USER(3): (if (listp '("Chicago" "Detroit" "Toronto"))
                "it is a list"
                "it ain't a list")
"it is a list"
CL-USER(4): (if (listp 99)
                "it is a list"
                "it ain't a list")
"it ain't a list"
```

3.2.6 Length of a List

Normally you can use the function `length` to get the number of elements in a list as an integer:

```
CL-USER(5): (length '(gm ford chrysler volkswagen))
4
CL-USER(6): (length nil)
0
```

The `length` function, as with most of Common Lisp, can itself be implemented in CL. Here is a simplified version of an `our-length` function which illustrates how `length` might be implemented:

```
(defun our-length (list)
  (if (null list)
      0
      (+ (our-length (rest list)) 1)))
```

Note that the function uses the symbol `list` to name its argument, which is perfectly valid as we discussed in section 3.1.4.

In English, this function says basically: “If the list is empty, its length is zero. Otherwise its length is one greater than the length of its rest.” As with many functions which operate on lists, this recursive definition is a natural way to express the length operation.

3.2.7 Member of a List

The `member` function will help you to determine whether a particular item is an element of a particular list. Like many similar functions, `member` uses `eq1` to test for equality, which is one of the most basic equality functions in CL (see Section 3.8.4 for a discussion of equality in CL). `Eq1` basically means the two objects must be the same symbol, integer, or actual object (i.e. the same address in memory).

`Member` takes two arguments: an item and a list. If the item is not in the list, it returns `NIL`. Otherwise, it returns the rest of the list, starting from the found element:

```
CL-USER(7): (member 'dallas '(boston san-francisco portland))
NIL
CL-USER(8): (member 'san-francisco '(boston san-francisco portland))
(SAN-FRANCISCO PORTLAND)
```

As with `length`, we could define `member` using a function definition which is very close to the English description of what the function is supposed to do³:

```
(defun our-member (elem list)
  (if (null list)
      nil
      (if (eql elem (first list))
          list
          (our-member elem (rest list))))))
```

In English, you might read this function to say “If the list is empty, return NIL. Otherwise, if the desired item is the same as the first of the list, return the entire list. Otherwise, do the same thing on the rest of the list.”

Note that, for the purposes of any kind of logical operators, returning any non-NIL value is “as good as” returning T. So the possible return-values of the `member` function are as good as returning T or NIL as far as any logical operators are concerned.

3.2.8 Getting Part of a List

`Subseq` is a common function to use for returning a portion of a list (or actually any type of Sequence). `Subseq` takes at least two arguments, a list and an integer indicating the position to start from. It also takes an optional third argument, an integer indicating the position to stop. Note that the position indicated by this third argument is not included in the returned sub-list:

```
CL-USER(9): (subseq '(a b c d) 1 3)
(B C)
CL-USER(10): (subseq '(a b c d) 1 2)
(B)
CL-USER(11): (subseq '(a b c d) 1)
(B C D)
```

Note also that the optional third argument in effect defaults to the length of the list.

³The use of the “nested” `if` statement in this example, while functionally correct, is a violation of good CL style. Later we will learn how to avoid nested `if` statements.

3.2.9 Appending Lists

The function **append** takes any number of lists, and returns a new list which results from appending them together. Like many CL functions, **append** does not side-effect, that is, it simply returns a new list as a return-value, but does not modify its arguments in any way:

```
CL-USER(6): (setq my-slides '(introduction welcome lists functions))
(INTRODUCTION WELCOME LISTS FUNCTIONS)
CL-USER(7): (append my-slides '(numbers))
(INTRODUCTION WELCOME LISTS FUNCTIONS NUMBERS)
CL-USER(8): my-slides
(INTRODUCTION WELCOME LISTS FUNCTIONS)
CL-USER(9): (setq my-slides (append my-slides '(numbers)))
(INTRODUCTION WELCOME LISTS FUNCTIONS NUMBERS)
CL-USER(10): my-slides
(INTRODUCTION WELCOME LISTS FUNCTIONS NUMBERS)
```

Note that the simple call to **append** does not affect the variable **my-slides**. If we wish to modify the value of this variable, however, one way to do this is by using **setq** in combination with the call to **append**. Note also the use of **setq** directly at the toplevel with a new variable name. For testing and “hacking” at the command prompt, this is acceptable, but in a finished program all such global variables should really be formally declared with **defparameter** or **defvar**.

3.2.10 Adding Elements to a List

To add a single element to the front of a list, you can use the function **cons**:

```
CL-USER(13): (cons 'a '(b c d))
(A B C D)
```

Cons is actually the low-level primitive function upon which many of the other list-constructing functions are built. As you may have guessed, “cons” stands for “CONStruct,” as in “construct a list.” When you read or hear people talking about a CL program doing a lot of “consing,” they are referring to the program’s behavior of building a lot of list structures, many of which are transitory and will need to be “freed up” by the automatic memory management subsystem, or “garbage collector.”

As with **append**, **cons** is non-destructive, meaning it does no side-effecting (modification) to its arguments.

3.2.11 Removing Elements from a List

The function `remove` takes two arguments, any item and a list, and returns a new list with all occurrences of the item taken out of it:

```
CL-USER(15): (setq data '(1 2 3 1000 4))
(1 2 3 1000 4)
CL-USER(16): (remove 1000 data)
(1 2 3 4)
CL-USER(17): data
(1 2 3 1000 4)
CL-USER(18): (setq data (remove 1000 data))
(1 2 3 4)
CL-USER(19): data
(1 2 3 4)
```

Like `append` and `cons`, `remove` is non-destructive and so does not modify its arguments. As before, one way to achieve modification with variables is to use `setq`.

3.2.12 Sorting Lists

The function `sort` will sort any sequence (including, of course, a list), based on comparing the elements of the sequence using any applicable comparison function, or predicate function. Because of efficiency reasons, the designers of CL made the decision to allow `sort` to modify (“recycle” the memory in) its sequence argument — so you must always “catch” the result of the sorting by doing something explicitly with the return-value:

```
CL-USER(20): (setq data '(1 3 5 7 2 4 6))
(1 3 5 7 2 4 6)
CL-USER(21): (setq data (sort data #'<))
(1 2 3 4 5 6 7)
CL-USER(22): (setq data (sort data #'>))
(7 6 5 4 3 2 1)
```

Notice that the comparison function must be a function which can take two arguments — any representative two elements in the given sequence — and compare them to give a T or NIL result. The “#” notation is a shorthand way of retrieving the actual function object associated with a symbol or expression (in this case, the `<` or `>` symbol). We will cover more on this in Section 3.4.

In the above examples, we simply reset the value of the variable `data` to the result of the `sort`, which is a common thing to do. If one wanted to retain the original pre-sorted sequence, a “safe” version of `sort` could be defined as follows:

```
(defun safe-sort (list predicate)
  (let ((new-list (copy-list list)))
    (sort new-list predicate)))
```

3.2.13 Treating a List as a Set

The functions `union`, `intersection`, and `set-difference` take two lists and compute the corresponding set operation. Because mathematical sets have no notion of ordering, the order of the results returned by these functions is purely arbitrary, so you should never depend on the results of these set operations being in any particular order:

```
CL-USER(23): (union '(1 2 3) '(2 3 4))
(1 2 3 4)
CL-USER(25): (intersection '(1 2 3) '(2 3 4))
(3 2)
CL-USER(26): (set-difference '(1 2 3 4) '(2 3))
(4 1)
```

3.2.14 Mapping a Function to a List

If you have one list, and desire another list of the same length, there is a good chance that you can use one of the mapping functions. `Mapcar` is the most common of such functions. `Mapcar` takes a function and one or more lists, and maps the function across each element of the list, producing a new resulting list. The term `car` comes from the original way in Lisp of referring to the first of a list (“contents of address register”). Therefore `mapcar` takes its function and applies it to the first of each successive rest of the list:

```
CL-USER(29): (defun twice (num)
              (* num 2))
TWICE
CL-USER(30): (mapcar #'twice '(1 2 3 4))
(2 4 6 8)
```

“Lambda” (unnamed) functions are used very frequently with `mapcar` and similar mapping functions. More on this in Section 3.4.3.

3.2.15 Property Lists

Property lists (“plists”) provide a simple yet powerful way to handle keyword-value pairs. A plist is simply a list, with an even number of elements, where each pair of elements represents a named value. Here is an example of a plist:

```
(:michigan "Lansing" :illinois "Springfield"
 :pennsylvania "Harrisburg")
```

In this plist, the keys are keyword symbols, and the values are strings. The keys in a plist are very often keyword symbols. keyword symbols are symbols whose names are preceded by a colon (:), and which are generally used just for matching the symbol itself (i.e. typically they are not used for their symbol-value or symbol-function).

For accessing members of a plist, CL provides the function `getf`, which takes a plist and a Key:

```
CL-USER(34): (getf '(:michigan "Lansing"
                    :illinois "Springfield"
                    :pennsylvania "Harrisburg")
                 :illinois)
"Springfield"
```

3.3 Control of Execution

“Control of execution” in CL boils down to “control of evaluation.” Using some standard and common macros, you control which forms get evaluated, among certain choices.

3.3.1 If

Perhaps the most basic operator for imposing control is the `if` operator, which we have already introduced briefly in Section 3.2.5. The `If` operator takes exactly three arguments, each of which is an expression which may be evaluated. The first is a test-form, the second is a then-form, and the third is an else-form (which may be left out for a default of `NIL`). When CL evaluates the `if` form, it will first evaluate the test-form. Depending on whether the return-value of the test-form is non-`NIL` or `NIL`, either the then-form or the else-form will be evaluated.

If you want to group multiple expressions together to occur as part of the then-form or the else-form, you must wrap them in an enclosing block. `Progn` is commonly used for this purpose. `Progn` will accept any number of forms as arguments, and will evaluate each of them in turn, finally returning the return-value of the last of them (see Figure 3.1).

```
(if (eql status :all-systems-go)
    (progn (broadcast-countdown) (flash-colored-lights)
           (play-eerie-music)(launch-rocket))
    (progn (broadcast-apology) (shut-down-rocket)))
```

Figure 3.1: Progn used with If

3.3.2 When

In some ways, `when` is similar to `if`, but you should use `when` in cases where you know that the else-clause is simply `NIL`. This turns out to be a common situation. Another advantage of using `when` in these cases is that there is no need for `progn` to group multiple forms — `when` simply takes a Test-form then any number of “Then-forms:”

```
CL-USER(36): (when (eql database-connection :active)
                  (read-record-from-database)
                  (chew-on-data-from-database)
                  (calculate-final-result))
999
CL-USER(37): (when (> 3 4)
                  (princ "I don't think so..."))
NIL
```

(Note that the database functions above are examples only and are not pre-defined in CL).

3.3.3 Logical Operators

The Logical Operators `and` and `or` evaluate one or more expressions given as arguments, depending on the return-values of the expressions. As we covered in Section 3.2.3, `T` is CL’s default representation for “True,” `NIL` (equivalent to the Empty List) is CL’s default representation for “False,” and any non-`NIL` value is “as good as” `T` as far as “Truth” is concerned:

```
CL-USER(38): (and (listp '(chicago detroit boston new-york))
                  (listp 3.1415))
NIL
CL-USER(39): (or (listp '(chicago detroit boston new-york))
                  (listp 3.1415))
T
```

In fact what is happening here is that the operator **and** evaluates its arguments (expressions) one at a time, from left to right, returning NIL as soon as it finds one which returns NIL — otherwise it will return the value of its last expression. **Or** evaluates its arguments until it finds one which returns non-NIL, and returning that non-NIL return-value if it finds one. Otherwise it will end up returning NIL.

Not takes a single expression as an argument and negates it — that is, if the expression returns NIL, **not** will return T, and if the argument returns non-NIL, **not** will return NIL. Logically, **not** behaves identically with the function **null** — but semantically, **null** carries the meaning of “testing for an Empty List,” while **not** carries the meaning of logical negation:

```
CL-USER(45): (not NIL)
T
CL-USER(46): (not (listp 3.1415))
T
```

3.3.4 Cond

Because the use of “nested” if statements is not appropriate, CL provides the macro **cond** to accomodate this. Use **cond** in situations when in other languages you might use a repeated “If...then...else if...else if...else if...” **Cond** takes as arguments any number of test-expression forms. Each of these forms consists of a list whose first is an expression to be evaluated, and whose rest is any number of expressions which will be evaluated if the first form evaluates to non-NIL. As soon as a non-NIL form is found, its corresponding expressions are evaluated and the entire **cond** is finished (i.e. there is no need for an explicit “break” statement or anything of that nature):

```
CL-USER(49): (let ((n 4))
              (cond ((> n 10) "It's a lot")
                    ((= n 10) "It's kind of a lot")
                    ((< n 10) "It's not a lot")))
"It's not a lot"
```

Because T as an expression simply evaluates to itself, a convenient way to specify a “default” or “catch-all” clause in a `cond` is to use T as the final conditional expression:

```
CL-USER(50): (let ((n 4))
               (cond ((> n 10) "It's a lot")
                     ((= n 10) "It's kind of a lot")
                     (t "It's not a lot")))
"It's not a lot"
```

3.3.5 Case

If you want to make a decision based on comparing a variable against a known set of constant values, `case` is often more concise than `cond`:

```
CL-USER(51): (let ((color :red))
               (case color
                 (:blue "Blue is okay")
                 (:red "Red is actually her favorite color")
                 (:green "Are you nuts?")))
"Red is actually her favorite color"
```

Note that `case` uses `eql` to do its matching, so it will only work with constants which are symbols (including keyword symbols), integers, etc. It will not work with strings.

CL also provides the macros `ecase` and `ccase`, which work just like `case` but provide some automatic error-handling for you in cases when the given value does not match any of the keys.

The symbol `otherwise` has special meaning to the `case` macro — it indicates a “default,” or “catch-all” case:

```
CL-USER(52): (let ((color :orange))
               (case color
                 (:blue "Blue is okay")
                 (:red "Red is actually her favorite color")
                 (:green "Are you nuts?")
                 (otherwise "We have never heard of that!")))
"We have never heard of that!"
```

3.3.6 Iteration

In addition to its innate abilities to do recursion and mapping, CL provides several operators for doing traditional iteration (“looping”). These include macros such as `do`, `dolist`, `dotimes`, and the “everything-including-the-kitchen-sink” iterating macro, `loop`. `Dolist` and `dotimes` are two of the most commonly used ones, so we will cover them here:

```
CL-USER(53): (let ((result 0))
               (dolist (elem '(4 5 6 7) result)
                 (setq result (+ result elem))))
22
```

As seen above, `dolist` takes an initialization-list and a body. The Initialization-list consists of an iterator-variable, a list-form, and an optional return-value-form (which defaults to `NIL`). The body will be evaluated once with the iterator-variable set to each element in the list which is returned by the list-form. When the list has been exhausted, the return-valueform will be evaluated and this value will be returned from the entire `dolist`. In other words, the number of iterations of a `dolist` is driven by the length of the list returned by its list-form.

`Dotimes` is similar in many ways to `dolist`, but instead of a list-form, it takes an expression which should evaluate to an integer. The body is evaluated once with the iteratorvariable set to each integer starting from zero (0) up to one less than the value of the Integer-form:

```
CL-USER(54): (let ((result 1))
               (dotimes (n 10 result)
                 (setq result (+ result n))))
46
```

In situations when you want the program to exit early from an iteration, you can usually accomplish this by calling `return` explicitly.

3.4 Functions as Objects

3.4.1 Named Functions

When working with CL, it is important to understand that a function is actually a kind of object, separate from any symbol or symbol-name with which it might be associated. You can access the actual function-object in a

symbol's function-slot by calling the function `symbol-function` on a symbol, like this:

```
(symbol-function '+)
#<Function +>
```

or this:

```
(symbol-function 'list)
#<Function LIST>
```

As you can see, the printed representation of a named function-object contains the function's symbol-name enclosed in pointy brackets, preceded by a pound sign.

You can explicitly call a function-object with the function `funcall`:

```
CL-USER(10): (funcall (symbol-function '+) 1 2)
3
CL-USER(11): (setq my-function (symbol-function '+))
#<Function +>
CL-USER(12): (funcall my-function 1 2)
3
```

A shorthand way of writing `symbol-function` is to write `#'` before the name of a function:

```
CL-USER(10): (funcall #' + 1 2)
3
CL-USER(11): (setq my-function #' +)
#<Function +>
CL-USER(12): (funcall my-function 1 2)
3
```

3.4.2 Functional Arguments

Functional arguments are arguments to functions which themselves are function-objects. We have seen this already with `mapcar`, `sort`, and `funcall`:

```
CL-USER(13): (defun add-3 (num)
              (+ num 3))
ADD-3
```

```
CL-USER(14): (symbol-function 'add-3)
#<Interpreted Function ADD-3>
CL-USER(15): (mapcar #'add-3 '(2 3 4))
(5 6 7)
CL-USER(17): (funcall #'add-3 2)
5
```

The idea of passing functions around to other functions is sometimes referred to as higher-order functions. CL provides natural support for this concept.

3.4.3 Anonymous Functions

Sometimes a function will be used so seldom that it is hardly worth going to the extent of creating Named function with `defun`. For these cases, CL provides the concept of the `lambda`, or “anonymous” (unnamed) function. To use a lambda function, you place the entire function definition in the position where you normally would put just the symbol which names the function, identified with the special symbol `lambda`:

```
CL-USER(19): (mapcar #'(lambda(num) (+ num 3))
                    '(2 3 4))
(5 6 7)
CL-USER(20): (sort '((4 "Buffy") (2 "Keiko") (1 "Judy") (3 "Aruna"))
                    #'(lambda(x y)
                        (< (first x) (first y))))
((1 "Judy") (2 "Keiko") (3 "Aruna") (4 "Buffy"))
```

3.4.4 Optional Arguments

Optional arguments to a function must be specified after any required arguments, and are identified with the symbol `&optional` in the argument list. Each optional argument can be either a symbol or a list containing a symbol and an expression returning a default value. In the case of a symbol, the default value is taken to be `NIL`:

```
CL-USER(23): (defun greeting-list (&optional (username "Jake"))
              (list 'hello 'there username))
GREETING-LIST
CL-USER(24): (greeting-list)
(HELLO THERE "Jake")
```

```
CL-USER(25): (greeting-list "Joe")  
(HELLO THERE "Joe")
```

3.4.5 Keyword Arguments

Keyword arguments to a function are basically named optional arguments. They are defined much the same as optional arguments:

```
(defun greeting-list (&key (username "Jake")  
                        (greeting "how are you?"))  
  (list 'hello 'there username greeting))
```

But when you call a function with keyword arguments, you “splice in” what amounts to a plist containing keyword symbols for the names of the arguments along with their values:

```
CL-USER(27): (greeting-list :greeting "how have you been?")  
(HELLO THERE "Jake" "how have you been?")  
CL-USER(28): (greeting-list :greeting "how have you been?" :username "Joe")  
(HELLO THERE "Joe" "how have you been?")
```

Note that with keyword arguments, you use a normal symbol (i.e. without the preceding colon) in the definition of the function, but a keyword symbol as the “tag” when calling the function. There is a logical reason behind this, which you will understand soon.

3.5 Input, Output, Streams, and Strings

Input and Output in CL is done through objects called streams. A stream is a source or destination for pieces of data which generally is connected to some physical source or destination, such as a terminal console window, a file on the computer’s hard disk, or a web browser. Each thread of execution in CL defines global parameters which represent some standard Streams:

- `*standard-input*` is the default data source.
- `*standard-output*` is the default data destination.
- `*terminal-io*` is bound to the console (command prompt) window, and by default this is identical to both `*standard-input*` and `*standard-output*`.

3.5.1 Read

Read is the standard mechanism for reading data items into CL. **Read** takes a single optional argument for its stream, and reads a single data item from that stream. The default for the stream is **standard-input**. If you simply type (**read**) at the command prompt, CL will wait for you to enter some valid Lisp item, and will return this item. **Read** simply reads, it does not evaluate, so there is no need to quote the data being read. You can set a variable to the result of a call to **read**:

```
(setq myvar (read))
```

The function **read-line** is similar to **read**, but will read characters until it encounters a new line or end-of-file, and will return these characters in the form of a string. **Read-line** is appropriate for reading data from files which are in a “line-based” format rather than formatted as Lisp expressions.

3.5.2 Print and Prin1

Print and **prin1** each take exactly one CL object and output its printed representation to a stream, which defaults to **standard-output**. **Print** puts a space and a newline after the item (effectively separating individual items with whitespace), and **Prin1** outputs the item with no extra whitespace:

```
CL-USER(29): (print 'hello)
```

```
HELLO  
HELLO
```

In the above example, you see the symbol **HELLO** appearing twice: the first time is the output actually being printed on the console, and the second is the normal return-value of the call to **print** being printed on the console by the read-eval-print loop. **Print** and **Prin1** both print their output readably, meaning that CL’s **read** function will be able to read the data back in. For example, the double-quotes of a string will be included in the output:

```
CL-USER(30): (print "hello")
```

```
"hello"  
"hello"
```

3.5.3 Princ

Princ, as distinct from **print** and **prin1**, prints its output in more of a human-readable format, which means for example that the double-quotes of a string will be stripped upon output:

```
CL-USER(31): (princ "hello")
hello
"hello"
```

in the above example, as before, we see the output twice: once when it is actually printed to the console, and again when the return-value of the call to **princ** is printed by the read-eval-print loop.

3.5.4 Format

Format is a powerful generalization of **prin1**, **princ**, and other basic printing functions, and can be used for almost all output. **Format** takes a Stream, a control-string, and optionally any number of additional arguments to fill in placeholders in the control-string:

```
CL-USER(33): (format t "Hello There ~a" 'bob)
Hello There BOB
NIL
```

In the above example, **t** is used as a shorthand way of specifying **standard-output** as the stream, and **~a** is a control-string placeholder which processes its argument as if by **princ**. If you specify either an actual stream or **T** as the second argument to **format**, it will print by side-effect and return **NIL** as its return-value. However, if you specify **NIL** as the second argument, **format** does not output to any stream at all by side-effect, but instead returns a string containing what would have been output to a stream if one had been provided:

```
CL-USER(34): (format nil "Hello There ~a" 'bob)
"Hello There BOB"
```

In addition to **~a**, there is a wealth of other format directives for an extensive choice of output, such as outputting items as if by **prin1**, outputting lists with a certain character after all but the last item, outputting numbers in many different formats including Roman numerals and English words, etc.

3.5.5 Pathnames

In order to name directories and files on a computer filesystem in an operating-system independent manner, CL provides the pathname system. CL pathnames do not contain system-specific pathname strings such as the forward-slash of Unix/Linux filenames or the backward-slash of MSDOS/Windows filenames. A CL pathname is basically a structure which supports a constructor function, `make-pathname`, and several accessor functions, such as `pathname-name`, `pathname-directory`, and `pathname-type`. The pathname structure contains six slots:

- Directory
- Name
- Type
- Device
- Version
- Host

On standard Unix filesystems, only the directory, name, and type slots are relevant. On MSDOS/Windows filesystems, the device slot is also relevant. On the Symbolics platform, which has a more advanced filesystem, the version and host slots also have relevance.

The constructor function, `make-pathname`, takes keyword arguments corresponding to the six possible slots of the pathname.

The following is an example of creating a pathname which would appear on Unix as `/tmp/try.txt`:

```
CL-USER(15): (defparameter *my-path*
                 (make-pathname :directory (list :absolute "tmp")
                               :name "readme"
                               :type "txt"))

*MY-PATH*
CL-USER(16): *my-path*
#p"/tmp/readme.txt"
```

As seen in the above example, the `:directory` slot is technically a list, whose first is a keyword symbol (either `:absolute` or `:relative`), and whose rest is the individual directory components represented as strings.

3.5.6 File Input and Output

Doing file input and output in CL is essentially a matter of combining the concepts of streams and pathnames. You must first open a file, which entails associating a stream with the file. CL does provide the `open` function, which directly opens a file and associates a stream to it. For a number of reasons, however, generally you will use a macro called `with-open-file` which not only opens the file, but automatically closes it when you are done with it, and performs additional cleanup and error-handling details for you. `With-open-file` takes a specification-list and a body. The specification-list consists of a stream-variable (a symbol) and a pathname, followed by several optional keyword arguments which specify, for example, whether the file is for input, output, or both. Here is an example of opening a file and reading one item from it (note that the file denoted by `*my-path*` must exist, or an error will be signalled. You can use the function `probe-file` to test for a file's existence.):

```
(with-open-file (input-stream *my-path*)
  (read input-stream))
```

No extra keyword arguments are required in the specification-list in this example, since “read-only” mode is the default for opening a file. In order to write to a file, however, additional keyword arguments must be given:

```
(with-open-file (output-stream *my-path*
                             :direction :output
                             :if-exists :supersede
                             :if-does-not-exist :create)
  (format output-stream "Hello There"))
```

3.6 Hash Tables, Arrays, Structures, and Classes

CL provides several other built-in data structures to meet a wide variety of needs. Arrays and hash tables support the storage of values in a manner similar to lists and plists, but with much faster look-up speed for large data sets. Structures provide another plist-like construct, but more efficient and structured.

Classes extend the idea of structures to provide a full object-oriented programming paradigm supporting multiple inheritance, and methods which

can dispatch based on any number of specialized arguments (“multimethods”). This collection of features in CL is called the Common Lisp Object System, or CLOS.

3.6.1 Hash Tables

A hash table is similar to what is known in some other languages as an associative array. It is comparable to a single-dimensional array which supports keys which are any arbitrary CL object, which will match using `eq1`, e.g. symbols or integers. Hash tables support virtually constant-time lookup of data, which means that the time it takes to look up an item in a hash table will remain stable, even as the number of entries in the hash table increases.

To work with hash tables, CL provides the constructor function `make-hash-table` and the accessor function `gethash`. In order to set entries in a hash table, use the `setf` operator in conjunction with the `gethash` accessor function. `Setf` is a generalization of `setq` which can be used with places resulting from a function call, in addition to exclusively with symbols as with `setq`. Here is an example of creating a hash table, putting a value in it, and retrieving the value from it:

```
CL-USER(35): (setq os-ratings (make-hash-table))
#<EQL hash-table with 0 entries @ #x209cf822>
CL-USER(36): (setf (gethash :windows os-ratings) :no-comment)
:NO-COMMENT
CL-USER(37): (gethash :windows os-ratings)
:NO-COMMENT
T
```

Note that the `gethash` function returns two values. This is the first example we have seen of a function which returns more than one value. The first return-value is the value corresponding to the found entry in the hash table, if it exists. The second return-value is a Boolean value (i.e. T or NIL), indicating whether the value was actually found in the hash table. If the value is not found, the first return-value will be NIL and the second return-value will also be NIL. This second value is necessary to distinguish from cases when the entry is found in the hash table, but its value just happens to be NIL.

There are several ways to access multiple return-values from a function, e.g. by using `multiple-value-bind` and `multiple-value-list`.

3.6.2 Arrays

Arrays are data structures which can hold single- or multi-dimensional “grids” full of values, which are indexed by one or more integers. Like hash tables, arrays support very fast access of the data based on the integer indices. You create arrays using the constructor function `make-array`, and refer to elements of the Array using the accessor function `aref`. As with hash tables, you can set individual items in the array using `setf`:

```
CL-USER(38): (setq my-array (make-array (list 3 3)))
#2A((NIL NIL NIL) (NIL NIL NIL) (NIL NIL NIL))
CL-USER(39): (setf (aref my-array 0 0) "Dave")
"Dave"
CL-USER(40): (setf (aref my-array 0 1) "John")
"John"
CL-USER(41): (aref my-array 0 0)
"Dave"
CL-USER(42): (aref my-array 0 1)
"John"
```

The `make-array` function also supports a myriad of optional keyword arguments for initializing the array at the time of creation.

3.6.3 Structures

Structures support creating your own data structures with named slots, similar to, for example, the `pathname` data structure which we have seen. Structures are defined with the macro `defstruct`, which supports many options for setting up the instance constructor functions, slots, and accessor functions of the structure. For a complete treatment of structures, we refer you to one of the CL references or to the on-line documentation for `defstruct`.

3.6.4 Classes and Methods

Classes and methods form the core of the Common Lisp Object System (CLOS), and add full-blown object-oriented capabilities to Common Lisp.

A complete treatment of CLOS is beyond the scope of this book, but we will provide a brief overview. Classes are basically “blueprints” or “prototypes” for objects. objects, in this context, are a kind of structure for grouping together both data and computational functionality. Methods are very similar to functions, but they can be specialized to apply to particular combinations of object types.

CLOS is a generic function object-oriented system, which means that methods exist independently from classes (i.e. methods do not “belong to” classes). Methods can take different combinations of class instance types as arguments, however, and can be specialized based upon the particular combinations of types of their arguments. Simple CL datatypes, such as strings and numbers, are also considered as classes.

Here is an example of defining a class and a method, making an instance of the class, and calling the method using the instance as the argument:

```
CL-USER(43): (defclass funbox () ((length :initform 10)
                                (width :initform 10)
                                (height :initform 10)))

#<STANDARD-CLASS FUNBOX>
CL-USER(44): (defmethod volume ((self funbox))
              (* (slot-value self 'length)
                 (slot-value self 'width)
                 (slot-value self 'height)))

#<STANDARD-METHOD VOLUME (FUNBOX)>
CL-USER(45): (setq mybox (make-instance 'funbox))
#<FUNBOX @ #x209d135a>
CL-USER(46): (volume mybox)
1000
```

As seen in the above example, the class definition takes at least three arguments: a symbol identifying the name of the class, a mixin-list which names superclasses (none, in this example), and a list of slots for the class, with default values specified by the `:initform` keyword.

The method definition is very much like a function definition, but its arguments (single argument in the case of this example) are specialized to a particular class type. When we call the method, we pass an instance of the class as an argument, using a normal argument list. This is unlike Java or C++, where a method is considered to be “of” a particular class instance, and only additional arguments are passed in an argument list.

CL is more consistent in this regard.

Because normal CL data types are also considered to be classes, we can define methods which specialize on them as well:

```
CL-USER(47): (defmethod add ((value1 string) (value2 string))
              (concatenate 'string value1 value2))

#<STANDARD-METHOD ADD (STRING STRING)>
```

```
CL-USER(48): (defmethod add ((value1 number) (value2 number))
              (+ value1 value2))
#<STANDARD-METHOD ADD (NUMBER NUMBER)>
CL-USER(49): (add "hello " "there")
"hello there"
CL-USER(50): (add 3 4)
7
```

3.7 Packages

Typically, symbols in CL exist within a particular package. You can think of packages as “area codes” for symbols. They are namespaces used within CL to help avoid name clashes. Package names are generally represented by keyword symbols (i.e. symbols whose names are preceded by a colon). ANSI CL has a “flat” package system, which means that packages do not “contain” other packages in a hierarchical fashion, but you can achieve essentially the same effect by “layering” packages. Actually, hierarchical packages have been added to Allegro CL as an extension to the ANSI Standard, and may be added to the ANSI standard at some point. However, the real use of hierarchical packages is to avoid name clashes with package names themselves. You can do this in any case simply by separating the components of a package name with a delimiter, for example, a dot: `:com.genworks.books`.

In a given CL session, CL always has a notion of the current package which is stored in the (dynamic) variable `*package*`. It is important always to be aware of what the current package is, because if you are in a different package than you think you are, you can get very surprising results. For example, functions that are defined in `:package-1` will not necessarily be visible in `:package-2`, so if you attempt to run those functions from `:package-2`, CL will think they are undefined.

Normally the CL command prompt prints the name of the current package. You can move to a different package, e.g. `foo`, with the toplevel command `:package :foo`.

Small programs can be developed in the `:user` package, but in general, a large application should be developed in its own package.

3.7.1 Importing and Exporting Symbols

Packages can export symbols to make them accessible from other packages, and import symbols to make them appear to be local to the package.

If a symbol is exported from one package but not imported into the current package, it is still valid to refer to it. In order to refer to symbols in other packages, qualify the symbol with the package name and a single colon: `package-2:foo`.

If a symbol is not exported from the outside package, you can still refer to it, but this would represent a style violation, as signified by the fact that you have to use a double-colon: `package-2::bar`.

3.7.2 The Keyword Package

We have seen many occurrences of symbols whose names are preceded by a colon (:). These symbols actually reside within the `:keyword` package, and the colon is just a shorthand way of writing and printing them. Keyword symbols are generally used for enumerated values and to name (keyword) function arguments. They are “immune” to package (i.e. there is no possibility of confusion about which package they are in), which makes them especially convenient for these purposes.

3.8 Common Stumbling Blocks

This section discusses common errors and stumbling blocks that new CL users often encounter.

3.8.1 Quotes

One common pitfall is not understanding when to quote expressions. Quoting a single symbol simply returns the symbol, whereas without the quotes, the symbol is treated as a variable:

```
CL-USER(6): (setq x 1)
1
CL-USER(7): x
1
CL-USER(8): 'x
X
```

Quoting a list returns the list, whereas without the quotes, the list is treated as a function (or macro) call:

```
CL-USER(10): '(+ 1 2)
(+ 1 2)
```

```
CL-USER(11): (first '(+ 1 2))
+
CL-USER(12): (+ 1 2)
3
CL-USER(13): (first (+ 1 2))
Error: Attempt to take the car of 3 which is not listp.
```

3.8.2 Function Argument Lists

When you define a function with `defun`, the arguments go inside their own list:

```
CL-USER(19): (defun square (num)
               (* num num))
SQUARE
```

But when you call the function, the argument list comes spliced in directly after the function name:

```
CL-USER(20): (square 4)
16
```

A common mistake is to put the argument list into its own list when calling the function, like this:

```
CL-USER(21) (square (4))
Error: Funcall of 4 which is a non-function.
```

If you are used to programming in Perl or Java, you will likely do this occasionally while you are getting used to CL syntax.

3.8.3 Symbols vs. Strings

Another point of confusion is the difference between symbols and strings. The symbol `AAA` and the string `"AAA"` are treated in completely different ways by CL. Symbols reside in packages; strings do not. Moreover, all references to a given symbol in a given package refer to the same actual address in memory — a given symbol in a given package is only allocated once. In contrast, CL might allocate new memory whenever the user defines a string, so multiple copies of the same string of characters could occur multiple times in memory. Consider this example:

```
CL-USER(17): (setq a1 'aaa)
AAA
CL-USER(18): (setq a2 'aaa)
AAA
CL-USER(19): (eq1 a1 a2)
T
```

EQL compares actual pointers or integers — A1 and A2 both point to the same symbol in the same part of memory.

```
CL-USER(20): (setq b1 "bbb")
"bbb"
CL-USER(21): (setq b2 "bbb")
"bbb"
CL-USER(22): (eq1 b1 b2)
NIL
CL-USER(23): (string-equal b1 b2)
T
```

Again, EQL compares pointers, but here B1 and B2 point to different memory addresses, although those addresses happen to contain the same string. Therefore the comparison by `string-equal`, which compares strings character-by-character, rather than the pointers to those strings, returns T.

Another difference between symbols and strings is that CL provides a number of operations for manipulating strings that do not work for symbols:

```
CL-USER(25): (setq c "Jane dates only Lisp programmers")
"Jane dates only Lisp programmers"
CL-USER(26): (char c 3)
#\e
CL-USER(27): (char 'xyz 1)
Error: 'XYZ' is not of the expected type 'STRING'
CL-USER(29): (subseq c 0 4)
"jane"
CL-USER(30): (subseq c (position #\space c))
" dates only lisp programmers"
CL-USER(31): (subseq c 11 (length c))
"only lisp programmers"
```

3.8.4 Equality

CL has several different predicates for testing equality. This is demanded because of the many different object types in CL, which dictate different notions of equality.

A simple rule of thumb is:

- Use `eql` to compare symbols, pointers (such as pointers to lists), and integers
- Use `=` to compare most numeric values
- Use `string-equal` for case-insensitive string comparison
- Use `string=` for case-sensitive string comparison
- Use `equal` to compare other types of objects (such as lists and single characters)
- For comparing floating-point numbers which may differ by a very small margin, but should still be considered equal, the safest technique is to take the absolute value of their difference, and compare this with an “epsilon” value within a tolerance of zero. In Figure 3.2 is a simple function which does this, which assumes that a global parameter is defined, `*zero-epsilon*`, as a numeric value very close to zero:

```
(defun nearly-equal? (num1 num2 &optional (tolerance *zero-epsilon*))
  (< (abs (- num1 num2)) tolerance))
```

Figure 3.2: Function for Floating-point Comparison

Examples:

```
CL-USER(33): (setq x :foo)
:FOO
CL-USER(34): (eql x :foo)
T
CL-USER(35): (= 100 (* 20 5))
T
CL-USER(36): (string-equal "xyz" "XYZ")
T
```

```
CL-USER(37): (setq abc '(a b c))
(A B C)
CL-USER(38): (equal abc (cons 'a '(b c)))
T
CL-USER(39): (equal #\a (char "abc" 0))
T
CL-USER(40): (eql abc (cons 'a '(b c)))
NIL
```

The last expression returns NIL because, although the two lists have the same contents, they reside in different places in memory, and therefore the pointers to those lists are different.

Note that several of the functions which by default use `eql` for matching will take an optional keyword argument `test` which can override this default:

```
CL-USER(54): (member "blue" '("red" "blue" "green" "yellow")
                  :test #'string-equal)
("blue" "green" "yellow")
```

Recall from Section 3.2.7 that `member` returns the rest of the list starting at the found element.

3.8.5 Distinguishing Macros from Functions

Since macros⁴ and functions both occur as the first element in an expression, at first glance it may appear difficult to distinguish them. In practice, this rarely becomes an issue. For beginners in the language, the best approach is to make a default assumption that something is a function unless you recognize it as a macro. Most macros will be readily identifiable because one of the following will apply:

- It is a familiar core part of CL, such as `if`, `cond`, `let`, `setq`, `setf`, etc.
- Its name begins with “def,” such as with `defun`, `defparameter`, etc.
- Its name begins with “with-,” as in `with-open-file`, `with-output-to-string`, etc.
- Its name begins with “do,” for example `dolist` and `dotimes`.

⁴Special Operators are one other category of operator in CL, which internally are implemented differently from macros, but which for our purposes we can consider like macros.

Beyond these general rules of thumb, if you find yourself in doubt about whether something is a function or a macro, you can:

1. Check with on-line reference documentation or in a CL reference book (see the Bibliography in Appendix B)
2. Use the `symbol-function` function to ask your CL session, like this:

```
CL-USER(24): (symbol-function 'list)
#<Function LIST>
CL-USER(25): (symbol-function 'with-open-file)
#<macro WITH-OPEN-FILE @ #x2000b07a>
```

3.8.6 Operations that cons

Because CL programs do not have to free and allocate memory explicitly, they can be written much more quickly. As an application evolves, some operations which “cons” can be replaced by operations which “recycle” memory, reducing the amount of work which the garbage collector (CL’s automatic memory management subsystem) has to do.

In order to write programs which conserve on garbage collecting, one technique is simply to avoid the unnecessary use of operations which cons. This ability will come with experience.

Another technique is to use destructive operations, which are best used only after one has gained more experience working with CL. Destructive operations are for the most part beyond the scope of this book.

Chapter 4

Interfaces

CL provides many mechanisms for interfacing with the “outside world.” This chapter will introduce some of the more important and practical ones available as part of Allegro CL, or as extensions to Allegro CL through third parties. These techniques include:

- Direct operating system calls
- Foreign Function Interface (FFI) for calling functions written in other languages
- CORBA
- TCP/IP Sockets
- Database interfaces
- Web Serving

4.1 The Operating System

4.1.1 System Calls

CL provides the function `excl:run-shell-command` for making direct calls to the operating system. Note that the `excl:` package qualifier is used, meaning that this call is an extension to Common Lisp and is specific to Allegro CL. Other CL platforms provide similar functionality. Calling OS commands directly is generally a convenient way of interfacing with the Unix (Linux) operating system or with legacy applications. It is also a quick way

to accomplish tasks such as file copying, moving, and deletion without using CL equivalents such as `rename-file` and `delete-file`.

`Run-shell-command` takes a string representing the desired command, and any optional keyword arguments as specified in the Allegro CL documentation.

```
CL-USER(2): (excl:run-shell-command "ls -l")
total 24
-rwxr-xr-x  1 dcooper users 2863 May  9 14:06 compile-system.cl
-rwxr-xr-x  1 dcooper users 1856 May  8 18:13 example.cl
-rw-r--r--  1 dcooper users  269 May  8 17:48 load-system.cl
-rw-r--r--  1 dcooper users 2152 May  8 18:10 readme.txt
-rwxr-xr-x  1 dcooper users  356 May  8 18:10 start.sh
0
CL-USER(3): (excl:run-shell-command "cp readme.txt readme.bak")
0
```

4.1.2 Environment Variables

To access values from Unix environment variables, you can use the function `sys:getenv`, which takes a string representing the name of the environment variable, and returns a string representing the current value (or `NIL` if the variable does not exist):

```
CL-USER(4): (sys:getenv "SHELL")
"/bin/bash"
```

You can also set the value of environment variables using `setf`:

```
CL-USER(5): (setf (sys:getenv "MYVAR") "yes")
"yes"
CL-USER(6): (sys:getenv "MYVAR")
"yes"
```

4.2 Foreign Function Interface

CL provides the capability to call functions written in other languages, most notably the C language. Each CL platform provides its own Foreign Function Interface (“FFI”), and the details of this are unfortunately not standardized between platforms. To learn the details of FFI for Allegro CL, you should consult Allegro CL’s documentation.

In general, the steps involved in calling foreign functions from CL are:

1. Define the Lisp names and types for the C functions and any supporting data structures
2. Load the shared library (`.so` file on Unix/Linux, or `.dll` on Windows) which contains the compiled object code for the functions
3. Call the functions

The FFI capability is extremely useful for “wrapping” existing libraries of C code, and/or for giving CL applications convenient access to low-level system calls. The Franz Inc. website contains documentation for a large library of Unix system calls which are pre-wrapped and available from CL.

4.3 CORBA

CORBA¹ is a standard for writing distributed object-oriented applications which can communicate over a network. A CORBA-wrapped application can be written in any language which supports CORBA. These applications can then communicate with each other, regardless of the language in which they are written.

CorbaLink is a CORBA implementation for Allegro CL, available through Franz Inc. or through third parties. Information on CORBA in general can be found at the Object Management Group’s website, <http://www.omg.org>. By using CorbaLink, CL applications can communicate in a seamless manner with applications written in other CORBA-supporting languages such as C++, Java, Smalltalk, and others.

4.4 TCP/IP Sockets

TCP/IP provides a standard for communication over a network. Most Internet communication is done via TCP/IP. Both servers and clients can be written which use TCP/IP as the underlying protocol.

In Allegro CL, the package `socket` provides CL-level functions which enable the use of TCP/IP. As a simple example, we will show how to write both a client and a server which can exchange data using TCP/IP sockets. These two programs can communicate when both running on the same machine, or they can run on separate machines anywhere on a TCP/IP network.

¹CORBA stands for Common Object Request Broker Architecture

First we write the server²:

```
(in-package :user)

(defparameter *socket-server-port* 4444)

(defun open-server ()
  (socket:make-socket :connect :passive
                     :local-port *socket-server-port*))

(defun test-socket-server ()
  (let ((socket (open-server)))
    (unwind-protect
      (let ((stream (socket:accept-connection socket)))
        (let ((command (read stream)))
          (cond ((eql command :hello)
                 (format stream "Hello to you!~%")
                 (finish-output stream))
                ((eql command :add-1-2)
                 (format stream "~a~%" (+ 1 2))
                 (finish-output stream))
                (t (format stream "Sorry, I don't understand~%")
                   (finish-output stream))))
          (close stream))
      (close socket)))))
```

Now we write a simple client, which will contact the server, send a command, and print out the response:

```
(in-package :user)

(defparameter *socket-server-port* 4444)

(defun test-socket-client (command)
  (let ((socket (socket:make-socket :remote-host "localhost"
                                     :remote-port *socket-server-port*)))
    (unwind-protect
```

²For space considerations we are not handling errors or trying to support multiple connections simultaneously. See the Allegro CL documentation for socket examples which do provide full error-handling and simultaneous connection support.

```
(progn
  (format socket "~s~%" command)
  (finish-output socket)
  (format t "~a~%" (read-line socket)))
(close socket)))
```

To test the server and client, start two CL processes. Load the server code into one process, and call

```
(test-socket-server)
```

in that process. Load the client code into the other process, and call

```
(test-socket-client :hello)
```

in that process. In the client, you should see the response from the server printed out. In the server, you should see the server return from the function call (since the server, as it is written, only serves one request, then exits). Note that you could change “localhost” in the definition of `test-socket-client` to any machine name on your network, and the client and server could communicate with each other between different machines.

Also note that both the `socket:make-socket` and `socket:accept-connection` functions accept several optional keyword parameters, which are documented in Allegro CL’s online and hardcopy manuals.

4.5 Databases

Allegro CL provides connectivity to many SQL-based database systems, such as Oracle, MySQL, PostgreSQL, Sybase and ODBC-compliant databases. Information on these packages, as well as more general information on accessing relational databases from CL, can be found at

<http://opensource.franz.com>

As a short example of database access from CL, we will present code for creating a table, adding some data to it, and querying the data with MySQL (which is a free open-source database server available on several platforms including Unix, Linux, and Windows).

```

(in-package :user)

(require :mysql)

(use-package :dbi.mysql)

(defun test-mysql ()
  (let ((db (connect "localhost" "testdb" "testuser" "testpasswd")))
    (sql db "create table captable (id int, name char(30))")
    (sql db "insert into captable values (~d, '~a')" 1 "Chicago")
    (sql db "insert into captable values (~d, '~a')" 2 "Austin")
    (sql db "insert into captable values (~d, '~a')" 3 "Philadelphia")
    (dolist (row (sql db "select * from captable where id > 1" :types :auto))
      (format t "~a: ~a~%" (first row) (second row)))
    (sql db "drop table captable")
    (disconnect db)))

```

In the above example, “testdb” is the name of a database which must exist on the MySQL server running on “localhost,” and “testuser” is a MySQL username which must have the proper access privileges to the database (“testpasswd” is the password).

Upon calling (`test-mysql`), we expect to see the following output:

```

2: Austin
3: Philadelphia

```

4.6 Webserving

Most CL vendors either provide direct support or provide third-party support for serving web content through their CL environments. For Allegro CL, Franz Inc. provides AllegroServe (“aserve”), a full-featured web application server. Applications built with aserve can provide all the benefits of traditional web applications, with the additional advantage that all state can be maintained in the running CL session. For traditional web applications, a new process must typically be started for each connection; this leads to an inherent inefficiency since all the application state must be fetched from a database or from persistent files every time.

AllegroServe can be obtained from

<http://opensource.franz.com>

and is distributed under an open-source (LLGPL³) license.

4.6.1 Publishing Static Pages

To start the webserver, you use the function `net.aserve:start`. It takes several keyword arguments. The only argument required is `:port`, which specifies the port number that the server should listen on. The default HTTP port is 80. If you are running as a normal (non-root) user on Unix/Linux, you will probably not have permission to run on port 80, so you should choose a number above 1024 and note it for use in forming your URLs. “8000” is a traditional “high port” default for running webserver. The URL for your webserver will then appear as `http://localhost:8000/` (of course replacing 8000 with whatever port number you actually use).

In order to serve a simple plain HTML (i.e. static) page from a file, the function is `publish-file`:

```
(net.aserve:start :port 8000)

(publish-file :path "/welcome.html" :file "/tmp/welcome.html")
```

The `:path` argument gives the URL path, while the `:file` argument gives the name in the Unix (or Windows) filesystem where the file actually resides.

To test the above example, create a simple HTML file and place it in `/tmp/welcome.html` (or use the `:file` argument with a different location as appropriate). A simple HTML file might look like this:

```
<html>
  <head>
    <title>Welcome</title>
  </head>
  <body>
    <h2>Welcome to my Web Page!</h2>
  </body>
</html>
```

Now, start your web browser and visit the page `http://localhost:8000/welcome.html`. You should see the web page come up in the browser.

³LLGPL is the Lisp Lesser General Public License

4.6.2 Publishing Dynamic Pages

Dynamic web page serving is more interesting, since a function is called to generate the HTML content on-the-fly. This function can tap into the entire running CL environment, to query data structures, call external databases, etc. Here is an example:

```
(publish :path "/hello"
        :content-type "text/html"
        :function
        #'(lambda(req ent)
            (with-http-response (req ent)
              (with-http-body (req ent)
                (html (:html
                      (:head (:title "Hello Page"))
                      (:body
                        ((:font :color "red")
                         (:h2 "Hello There!"))))))))))))
```

To test the above example, visit the URL `http://localhost:8000/hello` in your browser. You should see the heading “Hello There!” in red in your browser window.

You should also see in your CL console that some information is being logged on each visit to the page.

Note the use of the `html` macro, which works somewhat like the `format` function. It takes a series of HTML “tags” represented as lists — the first element of each list is a keyword symbol representing the “name” of the tag, and the rest of the list are either strings representing actual content, or more nested lists, or both.

4.6.3 Query Parameters

Query parameters are named values which can be spliced into a URL to pass data into the page. Within AllegroServe they can be accessed with the `request-query-value` function. As an example, consider the following publish form:

```
(publish :path "/welcome"
        :content-type "text/html"
        :function
        #'(lambda(req ent)
```

```

(with-http-response (req ent)
  (with-http-body (req ent)
    (let ((name (or (request-query-value "name" req)
                    "Pal"))))
      (html (:html
              (:head (:title "Welcome Page"))
              (:body
                ((:font :color "red")
                 (:h2 "Welcome, " (:princ name) "!"))))))))))

```

To test this, visit <http://localhost:8000/welcome?name=Joe> in your browser. You should see the heading “Welcome, Joe!” in the browser window.

4.6.4 Form Parameters

Query parameters are typically used for small amounts of data. For larger amounts of data, you will probably want to use HTML form parameters. These are generally filled in by a user through an interactive HTML form. To demonstrate the server side of such interaction, we will use the `curl` Unix command to simulate the posting of form data to a published aserve page.

Here is a `publish` form which can accept two form parameters named “color” and “shape”:

```

(publish :path "/form-test"
  :content-type "text/html"
  :function
  #'(lambda (req ent)
    (with-http-response (req ent)
      (with-http-body (req ent)
        (let ((color (request-query-value "color" req))
              (shape (request-query-value "shape" req)))
          (html (:html
                  (:head (:title "Form Test"))
                  (:body
                    (:h2 "You posted:")
                    (:princ (format nil "Color: ~a~%" color))
                    (:br)
                    (:princ (format nil "Shape: ~a" shape))))))))))

```

Now, from a Unix terminal window, type the following command:

```
curl --data "color=red&shape=circle" http://localhost:8000/form-test
```

You should see the following HTML printed on your console:

```
<html><head><title>Form Test</title></head>
<body><h2>You posted:</h2>Color: red<br />Shape: circle</body></html>
```

Note that `request-query-value` can be used to retrieve both URL-encoded query parameters and form parameters — aserve makes these available through the same interface.

4.6.5 Multipart Form Data and File Uploads

In order to handle binary files which are uploaded to the server, aserve provides the function `get-multipart-content`. Combined with the AllegroServe documentation and/or the aserve source code, this should be enough to get you started with handling uploaded files.

4.6.6 Development and Production Modes

AllegroServe has a concept of “development” and “production” mode which can be switched with the `:debug` keyword argument. In debug mode, when an error is detected the handler will stop with a break (see Section 2.8.3). This is usually convenient for development. In production mode, on the other hand, aserve will simply output a standard HTTP error response back to the browser and will not stop the server process.

The default debug setting for aserve is production mode. For development, you will typically want to start aserve with:

```
(net.aserve:start :port 8000 :debug t)
```

4.6.7 Other Features

This chapter has barely scratched the surface of what is available in AllegroServe. For a complete treatment, see the documentation available at the AllegroServe website listed above. Additional capabilities of note include:

- Authorization and security
- Support for SSL (Secure Sockets Layer)
- HTTP proxying

- Virtual hosting (multiple hostnames and DNS names being served by the same physical server)
- Support for client-side HTTP from CL
- Support for internationalization (international character sets) in URLs
- Support for serving WebDAV (Web-based Distributed Authoring and Versioning)

4.7 Embedded Languages

This section applies to layered application development. In particular, we cover the concept of an Embedded Language: a collection of macros and functions which add primitives and other key operators to CL, effectively creating a “language on top of CL.”

An example of this is GDL (Generative Design Language), which is the open-source core of the Genworks® GDL commercial product. GDL is a general-purpose language for Knowledge-Based Engineering, and is a significant superset of ANSI CL. Objects in GDL are similar to CLOS objects, but add features such as cached demand-driven behavior, automatic dependency tracking, and optional default values for unspecified data. GDL adds new primitives to CL, such as the `define-object` operator, which is analogous to CLOS’s `defclass`.

For layered applications and embedded languages, CL provides an environment unmatched by other current languages. Because CL allows macros to generate other macros, and the Lisp code can compute and generate Lisp code at all levels, it is straightforward to build extremely high-level languages on top of CL. And because of the CL-based syntax, these layered languages integrate seamlessly into the underlying CL environment.

4.8 Conclusion

Chapter 4 has provided a brief introduction into some of the interfacing capabilities available in CL. For additional information on any of these topics, you should refer to the on-line and printed documentation from Franz Inc., as well as to various resources available on the Web.

Chapter 5

The Squeakymail Example

This chapter will present a simple yet complete Common Lisp application. The application is a web-based email client, called “Squeakymail,” which can be used for reading, composing, sending, and managing email through a web browser. The complete code for this example can be downloaded from

`http://www.genworks.com/downloads`

In this chapter, we will give an overview of the application with excerpts from the source code, to help tie together the concepts covered in earlier chapters. This example is intended to give a flavor of working with CL on a real application.

5.1 Requirements and Design Overview

The basic requirements for this application are the following:

- Must be accessible from any web browser
- Must handle POP3 mailboxes
- Should allow the viewing of mail messages
- Should allow the composition and sending of mail
- Should allow the deletion and management of messages

Fortunately, Franz Inc. provides a free open-source library for handling POP3 email and for composing and sending mail. This library can also be downloaded from the above URL.

The webserver will use AllegroServe, as introduced in Chapter 4. The overall design breaks down as follows:

1. The AllegroServe session will store a list of messages in a global parameter
2. A background thread will periodically poll the POP3 server and update the list with any new mail
3. The web interface will provide pages to view the list of messages, to read the content of individual messages, and to compose and send new mail
4. Sufficient security will be in place to prevent multiple users of the application from reading each other's email

5.2 Main Application File

The main file for the application looks like this:

```
(in-package :user)

(defparameter *squeaky-port* 9876)

(defparameter *imap-server* "localhost")
(defparameter *pop-server* "localhost")
(defparameter *smtp-server* "localhost")

(defparameter *username* (sys:user-name))

(defparameter *poll-interval* 30)

(defun start-squeaky (&key (port *squeaky-port*))
  (start-server port)
  (start-background-polling))

(defun stop-squeaky ()
  (stop-background-polling)
  (stop-server))
```

The function `start-squeaky` starts the webserver and kicks off a background thread to poll the mailbox server for new mail. The `stop-squeaky` function stops this background thread and shuts down the webserver.

5.3 Working With Global State

Because the web application runs in a persistent CL session, the entire program state can be maintained in memory for fast and convenient access. No need to maintain state in a database or in some transient or persistent file system.

For this application, we maintain the state in three main global variables:

```
(defparameter *lock* (mp:make-process-lock))

(defparameter *messages* nil)

(defparameter *pollingp* nil)
```

The `*lock*` variable is a process lock, used for controlling access to the `*messages*` list in a multi-threaded environment. The `*messages*` variable contains a list of mail-message objects. The `*pollingp*` variable serves as a Boolean flag indicating whether the background polling process is currently running.

5.4 Background Polling

The `start-background-polling` and `stop-background-polling` functions spawn and halt a background thread which periodically polls for new mail:

```
(defun start-background-polling ()
  (setq *pollingp* t)
  (mp:process-run-function "Polling Process"
    #'polling-function))

(defun stop-background-polling ()
  (setq *pollingp* nil))

(defun polling-function ()
  (do () ((not *pollingp*))
    (update-messages-from-server))
```

```
(sleep *poll-interval*))
```

The call to `mp:process-run-function` spawns a new operating system thread. This function takes a name for the thread, and a function which the thread should invoke. The polling thread simply calls the `update-messages-from-server` function in a loop, waiting `*poll-interval*` seconds between each iteration.

The `update-messages-from-server` function is the function which actually contacts the POP3 server:

```
(defun update-messages-from-server ()
  (mp:with-process-lock (*lock*)
    (let ((count (ignore-errors (pop:pop-message-count *pop-server*
                                                         *username*)))))

      (when (and count (numberp count))
        (dotimes (n count)
          (unless (find n *messages* :key #'message-number)
            (let ((message (retrieve-message (1+ n))))
              (push message *messages*))))))))))
```

The `mp:with-process-lock` macro ensures that only one thread at a time can be accessing and modifying the `*messages*` list.

The `pop:pop-message-count` function returns the number of messages currently waiting on the server for the given username. The username in this example defaults to the Unix login name for the user running the CL session. You will have to ensure that you can authenticate yourself to your POP3 server with this username.

If there are messages on the server which are not yet in the `*messages*` list, they are retrieved and added to the list. The details of the `retrieve-message` function, which wraps the calls to retrieve the actual message, are not shown.

5.5 Serving the Web Pages

The application provides three main web pages:

1. Index — shows a list of the available messages, with links to read them
2. Read — shows the content of an individual message
3. Compose — allows the user to compose and send a new message

Here is the function which publishes the main index page:

```

(publish :path "/squeaky"
  :content-type "text/html"
  :function
  #'(lambda (req ent)
    (with-http-response (req ent)
      (with-http-body (req ent)
        (html
          (:html
            (:head (:title "Squeakymail"))
            (:body
              (:h1 "Your Mail")
              ((:form :method "post" :action "/squeaky/compose"
                ((:input :type "submit" :value "Compose New"))))
              (:hr)
              (if *messages*
                (html
                  ((:table :border 1)
                    (:tr (:th "Number") (:th "From")
                      (:th "Subject") (:th "Date")))
                    (dolist (message (reverse *messages*))
                      (html
                        (:tr
                          (:td ((:a :href
                            (format nil "/squeaky/read?number=~a"
                              (message-number message)))
                              (:princ (message-number message))))
                          (:td (:princ (message-from message)))
                          (:td (:princ (message-subject message)))
                          (:td (:princ (message-date message))))))
                        (html (:i "No messages"))))
                    (:hr)))))))))

```

This function uses the `html` macro from AllegroServe to generate the HTML content dynamically. The page includes a button to compose new mail, and a table showing all the messages currently in the `*messages*` list. Each message number is a hyperlink to read that particular message.

The “read” page is published in a similar manner:

```

(publish :path "/squeaky/read"
  :content-type "text/html"

```

```

:function
#' (lambda (req ent)
  (let* ((number-string (request-query-value "number" req))
        (number (when number-string
                     (ignore-errors (parse-integer number-string))))
        (message (when number
                     (find number *messages*
                           :key #'message-number))))
    (with-http-response (req ent)
      (with-http-body (req ent)
        (if message
          (html
            (:html
              (:head (:title "Read Mail"))
              (:body
                (:h2 "Message " (:princ number))
                (:b "From: ") (:princ (message-from message)) (:br)
                (:b "Subject: ") (:princ (message-subject message)) (:br)
                (:b "Date: ") (:princ (message-date message)) (:br)
                (:hr)
                (:pre (:princ (message-body message)))
                (:hr)
                ((:form :method "post" :action "/squeaky/delete")
                  ((:input :type "hidden" :name "number"
                           :value (format nil "~a" number)))
                  ((:input :type "submit" :value "Delete")))
                  ((:a :href "/squeaky") "Back to Index")))))
          (html
            (:html
              (:head (:title "Error"))
              (:body
                (:h2 "Message not found")
                ((:a :href "/squeaky") "Back to Index"))))))))

```

This page retrieves the message number from the query string, finds the corresponding message in the **messages** list, and displays its content. It also provides a button to delete the message and a link back to the main index.

The compose page is slightly more complex, as it needs to handle both displaying a blank form and processing the submitted form data:

[illegible]

```

:onclick "window.location='/squeaky'"))))
(:hr))))))))))

```

If the form has been submitted (i.e., if the “to,” “subject,” and “body” parameters are present), the `send-smtp-mail` function is called to send the message. Otherwise, a blank form is displayed.

5.7 Deleting Messages

The delete function removes a message from the local list and also deletes it from the POP3 server:

```

(publish :path "/squeaky/delete"
:content-type "text/html"
:function
#' (lambda (req ent)
  (let* ((number-string (request-query-value "number" req))
        (number (when number-string
                     (ignore-errors (parse-integer number-string)))))
    (when number
      (mp:with-process-lock (*lock*)
        (setq *messages* (remove number *messages*
                                  :key #'message-number))

        (ignore-errors
          (pop:delete-letter *pop-server* *username* number))))
    (with-http-response (req ent)
      (with-http-body (req ent)
        (html
          (:html
            (:head (:title "Message Deleted"))
            (:body
              (:h2 "Message has been deleted")
              ((:a :href "/squeaky") "Back to Index"))))))))

```

Note the use of `mp:with-process-lock` to ensure thread-safe modification of the `*messages*` list.

5.8 Starting the Webserver

The `start-server` and `stop-server` functions are simple wrappers around the AllegroServe functions:

```
(defun start-server (port)
  (net.aset:serve :start :port port))

(defun stop-server ()
  (net.aset:shutdown))
```

5.9 Message Data Structure

The application uses a simple structure to represent mail messages:

```
(defstruct message
  number
  from
  subject
  date
  body)
```

The `defstruct` macro automatically creates constructor, accessor, and type-predicate functions for this structure.

5.10 Testing the Application

To test Squeakymail, you will need:

1. A running POP3 mail server (or access to one)
2. A running SMTP mail server (or access to one)
3. The Post Office library from Franz Inc.
4. AllegroServe

Once these components are in place, you can start the application by loading all the source files and calling:

```
(start-squeaky)
```

Then point your web browser to:

`http://localhost:9876/squeaky`

You should see the main index page with your mail messages listed. From there you can read, compose, and delete messages.

5.11 Security Considerations

This example application is intended for demonstration purposes only and should not be used in a production environment without significant security enhancements. In particular:

- The application does not implement user authentication
- Passwords are not encrypted
- There is no session management to prevent unauthorized access
- The application assumes a single user

For a production application, you would need to implement proper authentication, session management, and access controls.

5.12 Enhancements

Some possible enhancements to this application might include:

- Support for IMAP in addition to POP3
- Ability to reply to messages
- Support for attachments
- Better formatting of HTML email
- Address book functionality
- Folders and message filtering
- Search functionality
- Support for multiple users with authentication

5.13 Conclusion

This chapter has demonstrated a complete, albeit simple, web application written in Common Lisp. The example shows how the various concepts covered in earlier chapters come together in a real application:

- Use of global parameters to maintain state

- Multi-threaded programming with process locks
- Web serving with AllegroServe
- Dynamic HTML generation
- External library integration (POP3/SMTP)
- Data structures and list manipulation

The complete source code for Squeakymail, along with installation instructions, can be downloaded from the URL given at the beginning of this chapter. We encourage you to download, install, and experiment with this application as a learning tool for Common Lisp development.

GDL/GWL Version of this Tutorial

This appendix describes the GDL version of this document. GDL is the open-source core of Genworks® GDL, and is a significant superset of ANSI CL. The GDL version of this tutorial is not currently available, but will be posted to

<http://www.genworks.com/downloads>

when it becomes available. The GDL version of this document will be similar to the CL version, but will include examples using GDL and GWL¹ (Generative Web Language) in addition to base CL.

GDL (Generative Design Language) is a domain-specific language extension to Common Lisp for knowledge-based engineering and product design applications. GDL objects extend CLOS objects with features such as:

- Cached, demand-driven computation
- Automatic dependency tracking
- Tree-structured object hierarchies
- Quantification (automatic generation of sequences of child objects)
- Optional default values for slots

GWL extends GDL to provide a declarative system for generating web-based user interfaces. GWL allows developers to specify web page layouts and interactions using GDL object definitions, automatically handling the HTTP request/response cycle and session management.

For more information about GDL and GWL, visit:

¹GWL stands for the Generative Web Language, an HTML templating system for GDL

<http://www.genworks.com>

The open-source Gendl system can be found at:

<http://www.gendl.org>

Bibliography

.1 Books

.1.1 Common Lisp: the Language, 2nd Edition

Guy L. Steele Jr., Digital Press, 1990, ISBN 1-55558-041-6

This is the book which initially defined the Common Lisp standard, before it was accepted by ANSI. Known affectionately as “CLtL2,” it remains the most comprehensive reference for ANSI CL. An on-line HTML version is available at:

<http://www.cs.cmu.edu/Groups/AI/html/cltl/cltl2.html>

.1.2 ANSI Common Lisp

Paul Graham, Prentice Hall, 1996, ISBN 0-13-370875-6

This is an excellent tutorial introduction to Common Lisp, covering all the important features of the language in a clear and practical style. Highly recommended for beginners.

.1.3 Practical Common Lisp

Peter Seibel, Apress, 2005, ISBN 1-59059-239-5

A hands-on introduction to Common Lisp that focuses on practical programming techniques and real-world applications. Includes several complete example programs. Also available online at:

<http://www.gigamonkeys.com/book/>

.1.4 Paradigms of Artificial Intelligence Programming: Case Studies in Common Lisp

Peter Norvig, Morgan Kaufmann, 1991, ISBN 1-55860-191-0

Known as “PAIP,” this book teaches both AI programming techniques and advanced Common Lisp programming through the development of significant AI systems. Essential reading for serious Common Lisp programmers.

.1.5 Object-Oriented Programming in Common Lisp: A Programmer’s Guide to CLOS

Sonya E. Keene, Addison-Wesley, 1989, ISBN 0-201-17589-4

The definitive guide to the Common Lisp Object System (CLOS). Covers classes, methods, generic functions, and the meta-object protocol.

.1.6 On Lisp: Advanced Techniques for Common Lisp

Paul Graham, Prentice Hall, 1993, ISBN 0-13-030552-9

An advanced book focusing on macros and other sophisticated Common Lisp programming techniques. Available online at:

<http://www.paulgraham.com/onlisp.html>

.1.7 Common Lisp Recipes

Edmund Weitz, Apress, 2016, ISBN 978-1-4842-1177-9

A cookbook of Common Lisp programming solutions, covering a wide range of practical problems and techniques.

.2 Online Resources

.2.1 Common Lisp HyperSpec

The ANSI Common Lisp specification in hypertext format, fully cross-referenced and searchable. The authoritative reference for the language:

<http://www.lispworks.com/documentation/HyperSpec/>

.2.2 The Common Lisp Cookbook

A collaborative project providing recipes and techniques for common programming tasks in Common Lisp:

<http://cl-cookbook.sourceforge.net/>

.2.3 Planet Lisp

An aggregator for Lisp-related blog posts and news:

<http://planet.lisp.org/>

.2.4 Quicklisp

A library manager for Common Lisp, making it easy to install and manage hundreds of libraries:

<http://www.quicklisp.org/>

.2.5 The Association of Lisp Users

The ALU maintains resources for the Lisp community, including a directory of Lisp systems:

<http://www.alu.org/>

.3 Tutorials

.3.1 Lisp Primer from Tulane University

An interactive online tutorial for beginners:

[http://www.cs.tulane.edu/www/Hennessy/courses/
cmps360/lisp_primer.html](http://www.cs.tulane.edu/www/Hennessy/courses/cmps360/lisp_primer.html)

.3.2 Common Lisp: A Gentle Introduction to Symbolic Computation

David S. Touretzky, available online:

<http://www.cs.cmu.edu/~dst/LispBook/>

.3.3 Successful Lisp

David B. Lamkins, available online:

<http://successful-lisp.blogspot.com/>

.4 Implementation-Specific Documentation

.4.1 Allegro CL Documentation

Complete documentation for Franz Inc.'s Allegro Common Lisp:

<http://www.franz.com/support/documentation/>

.4.2 SBCL Manual

Documentation for Steel Bank Common Lisp:

<http://www.sbcl.org/manual/>

.4.3 CCL Documentation

Documentation for Clozure Common Lisp:

<http://ccl.clozure.com/manual/>

.5 Community and Discussion

.5.1 comp.lang.lisp

The Usenet newsgroup for Lisp discussion, accessible via Google Groups or news readers.

.5.2 Reddit /r/lisp

An active community for Lisp discussion:

<http://www.reddit.com/r/lisp/>

.5.3 Lisp Discord and IRC Channels

Active real-time chat communities for Lisp programmers can be found on Discord and IRC (`#lisp` on Libera.Chat).

Emacs Customization

This appendix provides some tips and techniques for customizing Emacs to work more effectively with Common Lisp.

.6 Key Bindings

The default Emacs key bindings use Control and Meta extensively. On most keyboards:

- **Control** is the key labeled “Ctrl” or “Control”
- **Meta** is typically the key labeled “Alt” or (on some systems) the Windows key

.6.1 Remapping Caps Lock

Many Emacs users find it convenient to remap the Caps Lock key to function as an additional Control key, since Control is used so frequently. This can significantly reduce hand strain for heavy Emacs users.

On Linux/X11

Add the following to your `.xinitrc` or `.xsession` file:

```
setxkbmap -option ctrl:nocaps
```

Or use `xmodmap` with a file `.xmodmaprc` containing:

```
clear Lock
keycode 66 = Control_L
add Control = Control_L
```

Then run:

```
xmodmap ~/.xmodmaprc
```

On Windows

Several utilities are available to remap keys on Windows, such as:

- SharpKeys (free, GUI-based)
- AutoHotkey (scriptable)
- Windows Registry editing (for advanced users)

On macOS

Go to System Preferences → Keyboard → Modifier Keys, and remap Caps Lock to Control.

.7 Essential Emacs Commands

Here are some essential Emacs commands that are particularly useful when working with Lisp:

.7.1 Navigation

- **C-a** — Move to beginning of line
- **C-e** — Move to end of line
- **M-<** — Move to beginning of buffer
- **M->** — Move to end of buffer
- **C-M-f** — Move forward by s-expression
- **C-M-b** — Move backward by s-expression
- **C-M-d** — Move down into s-expression
- **C-M-u** — Move up out of s-expression

.7.2 Editing

- **C-k** — Kill (cut) to end of line
- **M-w** — Copy region
- **C-w** — Kill (cut) region

- C-y — Yank (paste)
- C-/ or C-_ — Undo
- C-M-k — Kill s-expression forward
- M-C-k — (alternate) Kill s-expression forward
- C-M-t — Transpose s-expressions

.7.3 Buffers and Windows

- C-x b — Switch to buffer
- C-x k — Kill (close) buffer
- C-x C-f — Find (open) file
- C-x C-s — Save file
- C-x C-w — Write file (save as)
- C-x 2 — Split window horizontally
- C-x 3 — Split window vertically
- C-x 1 — Delete other windows
- C-x 0 — Delete current window
- C-x o — Switch to other window

.7.4 Lisp-Specific Commands

When using the Emacs-Lisp interface (as described in Chapter 2):

- C-M-x — Compile and load the current top-level form
- C-c C-b — Compile and load the current buffer
- M-A — Show argument list for function at point
- C-c . — Find definition of symbol at point
- M-, — Return from definition lookup
- C-c C-k — Compile and load file
- C-c C-z — Switch to Lisp listener

.8 Customizing .emacs

The `.emacs` file in your home directory is executed when Emacs starts up. This file can contain Emacs Lisp code to customize your Emacs environment.

.8.1 Basic .emacs Example

Here is a simple `.emacs` file with some useful settings:

```
;; Load the Allegro CL\index{Allegro CL} - Emacs interface
(setq load-path (cons "/usr/local/acl62/eli" load-path))
(load "fi-site-init")

;; Don't show the startup message
(setq inhibit-startup-message t)

;; Show line and column numbers
(setq line-number-mode t)
(setq column-number-mode t)

;; Enable syntax highlighting
(global-font-lock-mode t)

;; Show matching parentheses
(show-paren-mode t)

;; Electric pair mode (auto-close parens)
(electric-pair-mode t)

;; Auto-revert buffers when files change on disk
(global-auto-revert-mode t)

;; Use spaces instead of tabs
(setq-default indent-tabs-mode nil)

;; Set tab width to 2 spaces
(setq-default tab-width 2)
```

.8.2 Keyboard Macro for Common Lisp Session

You can define custom key bindings to make frequent operations more convenient:

```
;; Quick key to switch to *common-lisp* buffer
(global-set-key (kbd "C-c l")
  (lambda ()
    (interactive)
    (switch-to-buffer "*common-lisp*")))

;; Quick key to start Common Lisp
(global-set-key (kbd "C-c C-l") 'fi:common-lisp)
```

.8.3 ParEdit Mode

ParEdit is a minor mode for Emacs that provides structured editing of S-expressions. It ensures that parentheses always remain balanced and provides commands for manipulating S-expressions structurally. Many experienced Lisp programmers find ParEdit indispensable.

ParEdit can be installed via MELPA (see Section .9).

.8.4 Rainbow Delimiters

Rainbow Delimiters mode colorizes nested parentheses with different colors, making it easier to see the structure of deeply nested code. This is particularly helpful when working with Lisp.

To use Rainbow Delimiters, install it via MELPA and add to your `.emacs`:

```
(require 'rainbow-delimiters)
(add-hook 'lisp-mode-hook 'rainbow-delimiters-mode)
(add-hook 'emacs-lisp-mode-hook 'rainbow-delimiters-mode)
```

.9 Package Management: MELPA

MELPA (Milkypostman's Emacs Lisp Package Archive) is a repository of Emacs packages that can be easily installed. To enable MELPA, add the following to your `.emacs`:

```
(require 'package)
(add-to-list 'package-archives
```

```
'("melpa" . "http://melpa.org/packages/") t)
(package-initialize)
```

After restarting Emacs, you can use M-x `package-list-packages` to browse and install packages.

.10 SLIME: An Alternative to the Emacs-Lisp Interface

SLIME (Superior Lisp Interaction Mode for Emacs) is an alternative to the Franz Inc. Emacs-Lisp interface. SLIME is a popular open-source project that works with many Common Lisp implementations, including SBCL, CCL, and Allegro CL.

SLIME can be installed via MELPA or Quicklisp. It provides features such as:

- Interactive debugging with a graphical debugger
- Code completion
- Cross-referencing
- Inspector for examining objects
- Profiling and tracing support

For more information, visit:

<http://common-lisp.net/project/slime/>

.11 Color Themes

Emacs supports color themes to customize the appearance of your editing environment. Many themes are available through MELPA, or you can create your own.

Popular themes include:

- solarized (light and dark versions)
- zenburn
- monokai

- `tomorrow` (light and dark versions)

To try a theme, install it via MELPA and use `M-x load-theme`.

.12 Other Useful Packages

.12.1 Company Mode

Company (Complete Anything) provides text completion. It works well with SLIME for Common Lisp code completion.

.12.2 Magit

Magit is a sophisticated interface to Git version control within Emacs. If you're using Git for your Lisp projects, Magit is highly recommended.

.12.3 Projectile

Projectile provides project management features, making it easy to navigate between files in a project, search within a project, and more.

.13 Learning More About Emacs

Emacs has extensive built-in documentation:

- `C-h t` — Start the Emacs tutorial
- `C-h k` — Describe key binding
- `C-h f` — Describe function
- `C-h v` — Describe variable
- `C-h m` — Describe current major and minor modes
- `C-h i` — Open Info documentation browser

For online resources, the Emacs Wiki (<http://www.emacswiki.org/>) contains a wealth of information, tips, and customization examples.

Afterword

We hope that this book has given you a solid foundation in Common Lisp and has inspired you to explore this powerful language further. Common Lisp represents one of the oldest continuously-used programming languages, yet it remains remarkably modern and relevant for today's software development challenges.

The Power of Lisp

The features that made Lisp powerful in the 1960s — dynamic typing, automatic memory management, interactive development, and code-as-data — are now recognized as essential features in modern programming languages. What sets Common Lisp apart is that it has had these features from the beginning, refined over decades of practical use.

The ability to write code that writes code, through macros and metaprogramming, gives Common Lisp a flexibility that few other languages can match. This power allows you to create domain-specific languages embedded within CL, effectively raising the level of abstraction at which you program.

The Lisp Community

One of the great strengths of Common Lisp is its community. Lisp programmers tend to be thoughtful, experienced developers who value elegance and power in their tools. The community maintains a wealth of open-source libraries, documentation, and shared knowledge.

We encourage you to participate in the community:

- Ask questions on forums and mailing lists
- Contribute to open-source Lisp projects

- Write about your experiences with Lisp
- Share your code and libraries
- Help newcomers learn the language

Continuing Your Journey

This book has covered the fundamentals, but there is much more to learn:

- Advanced macro techniques
- The Common Lisp Object System (CLOS) in depth
- The condition system for sophisticated error handling
- The reader macro system for extending CL syntax
- Practical application development and deployment
- Domain-specific languages built on CL
- Performance optimization techniques

The books and resources listed in Appendix B will help you continue your learning. We particularly recommend Paul Graham’s “On Lisp” for advanced macro programming, and Peter Norvig’s “Paradigms of Artificial Intelligence Programming” for substantial examples of CL in practice.

Common Lisp in Industry

Common Lisp is used in production systems around the world, often in domains where reliability, flexibility, and sophisticated problem-solving are critical:

- Knowledge-based engineering and CAD systems
- Financial trading and risk analysis
- Scheduling and optimization
- Natural language processing
- Bioinformatics

- Web applications
- Aerospace and defense

While Common Lisp may not have the market share of languages like Java or Python, it has a dedicated user base and continues to be the language of choice for applications where its unique strengths matter.

The Future of Common Lisp

Common Lisp's strength lies in its stability. The ANSI standard ensures that code written today will continue to work decades from now. At the same time, implementations continue to improve, adding modern features like:

- Better integration with modern operating systems
- Improved performance on current hardware
- Support for contemporary protocols and formats
- Enhanced development tools and IDEs
- Growing libraries of open-source code

The emergence of the Quicklisp library manager has made it easier than ever to discover and use Common Lisp libraries. The continued development of implementations like SBCL, CCL, and Allegro CL ensures that Common Lisp remains a viable choice for serious software development.

Final Thoughts

Programming in Common Lisp is different from programming in most other languages. The interactive, incremental development style can feel strange at first if you're used to the edit-compile-link-run cycle of languages like C++ or Java. But once you become comfortable with the Lisp way of working, it's hard to go back.

The REPL (Read-Eval-Print Loop) isn't just a way to test small code snippets — it's a window into a living, breathing program that you're building and modifying in real time. The ability to redefine functions, experiment with data structures, and test ideas immediately, all while your program continues to run, provides a development experience that is difficult to match in other languages.

We encourage you to embrace the Lisp philosophy: start simple, build incrementally, and let the language adapt to your problem rather than forcing your problem into the constraints of the language.

Acknowledgments

This book would not have been possible without the contributions of many people over the years:

- The designers and implementers of Common Lisp, who created a language that has stood the test of time
- The Common Lisp community, who have shared their knowledge and code freely
- Franz Inc., for their continued support of Common Lisp and their contributions to the community
- The many authors who have written books and tutorials about Lisp, making the language accessible to new generations of programmers

Closing

Thank you for reading this book. We hope it has been helpful in your journey to learn Common Lisp. Whether you're using CL for professional development, academic research, or personal projects, we wish you success and enjoyment in your Lisp programming.

Remember: Lisp is not just a programming language — it's a way of thinking about computation. Once you truly understand Lisp, you'll see programming differently, regardless of what language you're using.

Happy Lisp-ing!

David J. Cooper, Jr.
February 14, 2011

Index

Allegro CL, 5, 57, 65, 94, 105
American National Standards Institute, 2
and, 43
ANSI Common Lisp, 2, 91
append, 39
aref, 55
arrays, 27
Association of Lisp Users, 93
associative array, 54
Automatic Memory Management, 3

car, 41
case, 45
ccase, 45
CGI scripts, 1
CLOS, 54, 75, 89, 92, 104
Common Lisp, 1, 5, 23, 65, 77, 91, 95, 103
Common Lisp Object System, 54, 92, 104
Computer-aided Software Engineering, 2
cond, 44
cons, 39
consing, 39
CORBA, 65

defclass, 75
defparameter, 31
defstruct, 55, 85
defun, 30

defvar, 31
destructive operators, 27
do, 46
dolist, 46
dotimes, 46
Dynamic Redefinition, 3
dynamic scope, 33
Dynamic Typing, 3

ecase, 45
Emacs, 8, 95
eq, 37
equal, 61

format, 51, 72
hash tables, 3, 27
if, 36

lambda, 48
let, 32
lexical scope, 33

make-array, 55
mapcar, 41
member, 37

not, 44
null, 35

optional arguments, 49
or, 43

princ, 51

print, 50

read, 50

remove, 40

setf, 54, 66

setq, 32

SLIME, 100

sort, 40

when, 43